MODULE 01

Object-Oriented Programming in Python

Classes, exceptions, inheritance, object design and the SOLID principles

```
>>> class Account: ...
```

AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering



Learning Objectives

By the end of this module, you will be able to:

1. Explain objects (state, behaviour, identity), classes, abstraction and encapsulation
2. Define classes with initializers, methods, properties and special (dunder) methods
3. Choose between class attributes, `@classmethod`, `@staticmethod` and `@dataclass`
4. Raise, handle and chain exceptions and design a custom exception hierarchy
5. Use inheritance, `super()`, the MRO, polymorphism and duck typing
6. Define interfaces with `abc.ABC` and `typing.Protocol`
7. Evaluate designs for cohesion and coupling; prefer composition and delegation
8. Apply the SOLID principles to refactor Python classes

MODULE 01 · OBJECT-ORIENTED PROGRAMMING

Module Roadmap

01 **Objects and Classes**
State, behaviour, identity — and the blueprints behind them

02 **Defining Classes in Python**
Initializers, attributes, methods, properties and special methods

03 **Exceptions and Custom Exception Classes**
Raising, handling, propagating and designing exceptions

04 **Inheritance and Polymorphism**
Reusing and specialising classes — and treating different objects alike

05 **Object Design Fundamentals**
Complexity, cohesion, coupling, composition and delegation

06 **SOLID Principles I**
Design smells, single responsibility and open/closed modules

07 **SOLID Principles II**
Substitutable subtypes, lean interfaces and inverted dependencies

01

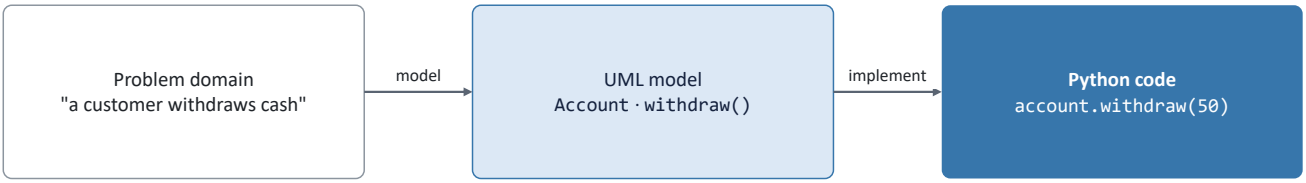
Objects and Classes

State, behaviour, identity — and the blueprints behind them

- Thinking in objects
- Objects: state, behaviour, identity
- Classes as blueprints
- Abstraction and encapsulation

Thinking in Objects

- We describe problems with **nouns** (things) and **verbs** (actions)
- OO models both the *problem domain* and the *solution domain* with objects
- **UML** (Unified Modeling Language) is the de facto notation for OO models
- OO languages keep the code close to our mental model



OO Concepts at a Glance

Building Blocks
- Objects
- Classes
- Interfaces

Four Pillars
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

Design Qualities
- Cohesion and coupling
- Associations and links
- Delegation

Where each concept is taught

Sections 1–2: building blocks · Section 3: exceptions · Section 4: inheritance and polymorphism · Section 5: design qualities · Sections 6–7: SOLID

What Is an Object?

Definition

"An object has state, behavior, and identity; the structure and behavior of similar objects are defined in their common class." — Booch, *Object Solutions*

State

The data it holds: attribute values

`balance = 3745.56`

Behaviour

What it can do: its methods

`account.deposit(100)`

Identity

Which object it is, independent of its state

`id(account)`, `a is b`

Every object is an instance of exactly one class: methods are shared by the class, attribute values belong to each object.

Everything in Python Is an Object

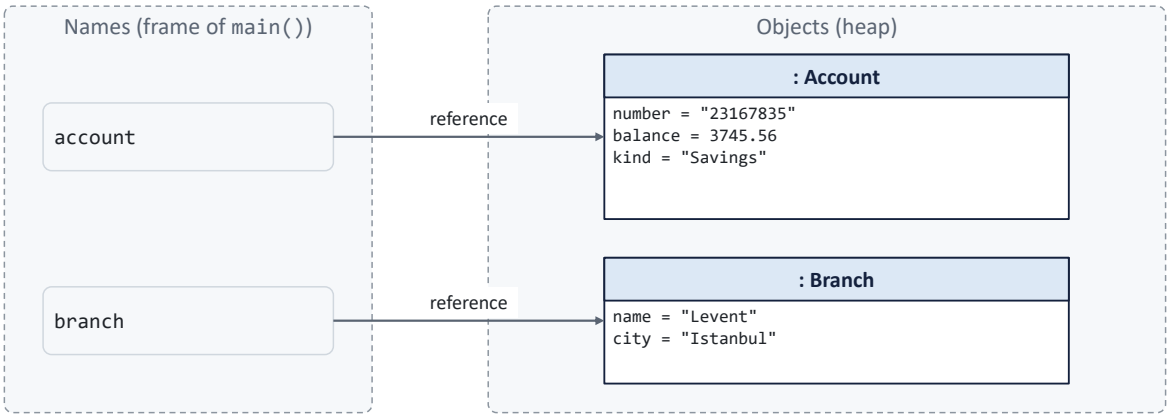
```
>>> balance = 3745.56
>>> type(balance)
<class 'float'>
>>> balance.is_integer()
False
>>> owners = ["Ada", "Alan"]
>>> alias = owners
>>> clone = list(owners)
>>> alias is owners, clone is owners
(True, False)
>>> clone == owners
True
>>> type(print)
<class 'builtin_function_or_method'>
```

Aspect	Ask Python
Identity	`id(obj)`, `a is b`
Type (class)	`type(obj)`, `isinstance(obj, C)`
State (value)	`a == b`, `vars(obj)`
Behaviour	`obj.method()`, `dir(obj)`

Key idea

`is` compares **identity**; `==` compares **values**.

Names Refer to Objects



Key idea

A variable stores a **reference**. `other = account` copies the reference, not the object: both names now refer to the same `Account`.

What Is a Class?

Definition

"A class is a blueprint or prototype from which objects are created." — *The Java Tutorials*, Oracle

A class provides…	In Python
attribute metadata	names (and type hints) set in `__init__`
method signatures	`def` statements in the class body
method implementations	the bodies of those methods
initialization	the **initializer** `__init__`

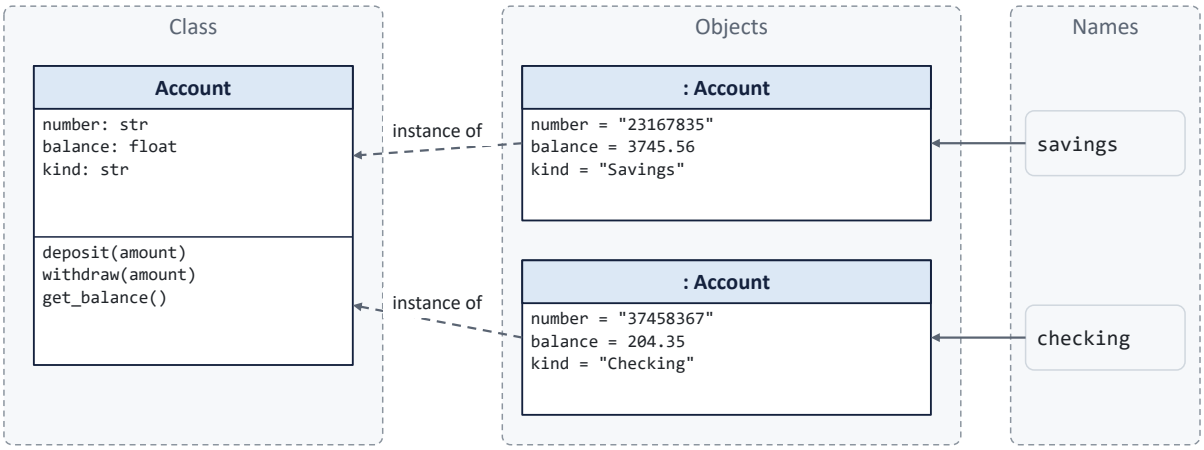
account.py

```python
class Account:
    """Blueprint for bank accounts."""

    def __init__(self, number: str,
                 balance: float) -> None:
        self.number = number
        self.balance = balance

    def deposit(self, amount: float) -> None:
        self.balance += amount
```

One Class, Many Objects

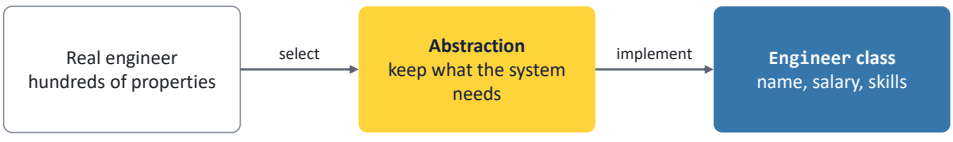


Key idea

Methods are stored **once** in the class; every object has its **own** attribute values.

Abstraction

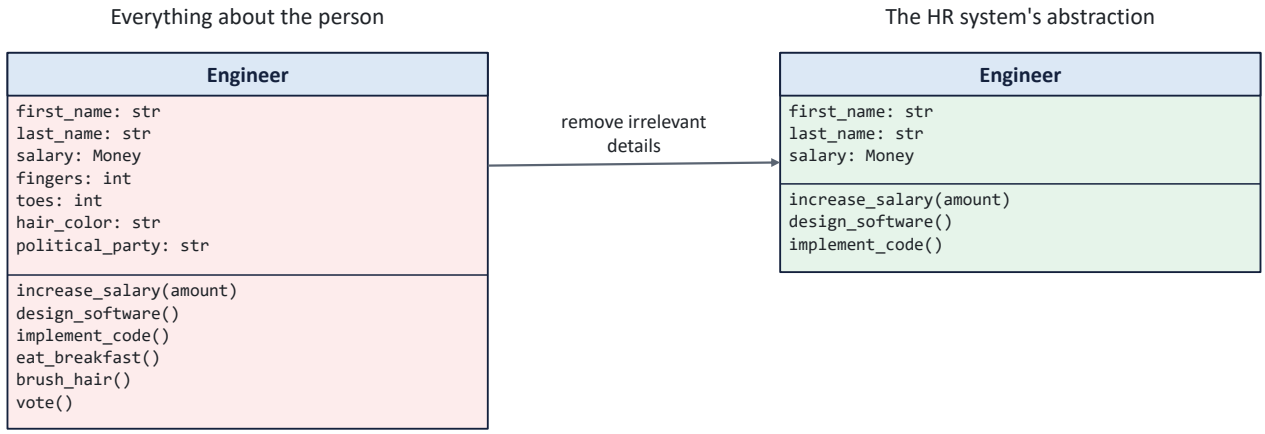
- **Abstraction**: a simplified but *relevant* view of a real-world thing
- Relevance depends on the problem and solution domains
- Details irrelevant to the *system* are **removed**
- Details irrelevant to the current *view* are **hidden**



Key idea

Model what the system needs — not everything that is true about the real object.

Abstraction: Choosing Relevant Details



Tip

Relevance depends on the system: a hospital system would keep blood type; a payroll system never would.

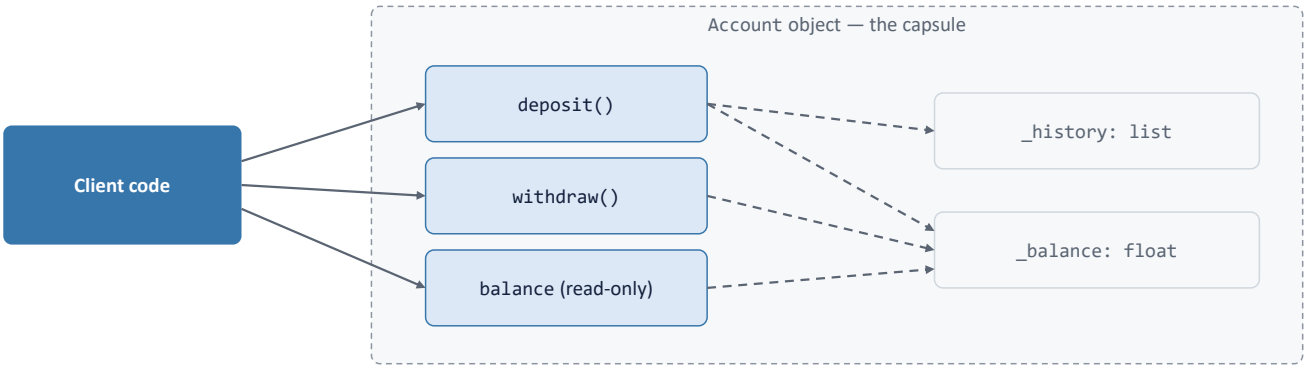
Encapsulation and Information Hiding

Encapsulation

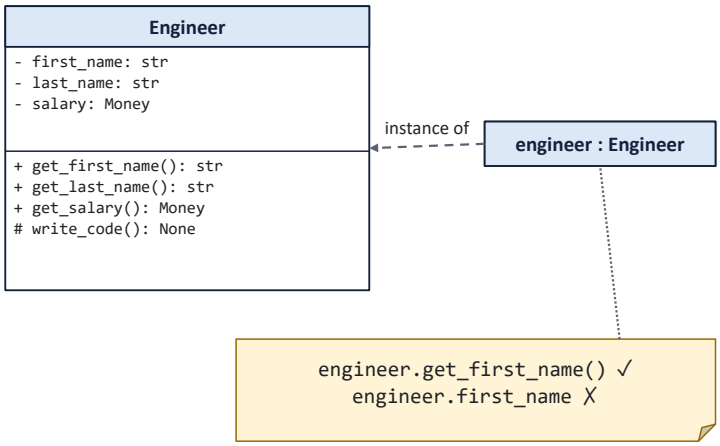
Bundle state and the methods that use it inside the object's boundary — “to enclose in or as if in a capsule” (Webster)

Information Hiding

Hide implementation details behind a set of non-private methods, so they can change without breaking clients



Visibility: UML Markers and Python Conventions



UML	Meaning	Python
`+`	public	`name`
`#`	protected (subclasses)	`_name` (convention)
`-`	private	`__name` (name mangling)

Tip

Python enforces none of these: it relies on conventions. Start with a public attribute; switch to a property when you need control.

QUIZ

Quick Check: Abstraction

An HR system's `Engineer` class keeps name and salary but not hair colour or shoe size. Which concept does this illustrate?

A. Encapsulation

B. Abstraction

C. Inheritance

D. Polymorphism

ANSWER

Quick Check: Abstraction

An HR system's `Engineer` class keeps name and salary but not hair colour or shoe size. Which concept does this illustrate?

- A Encapsulation
- **B Abstraction**
- C Inheritance
- D Polymorphism

Why

Abstraction keeps only the details that are relevant to the system. Encapsulation bundles the chosen state with its methods and hides it behind an interface.

TRACE THE CODE

Aliases and Copies

```python
owners = ["Ada", "Alan"]
alias = owners
backup = list(owners)
alias.append("Grace")

print(owners)
print(backup)
print(alias is owners, backup == owners)
```

Predict the three lines of output. How many list objects exist?

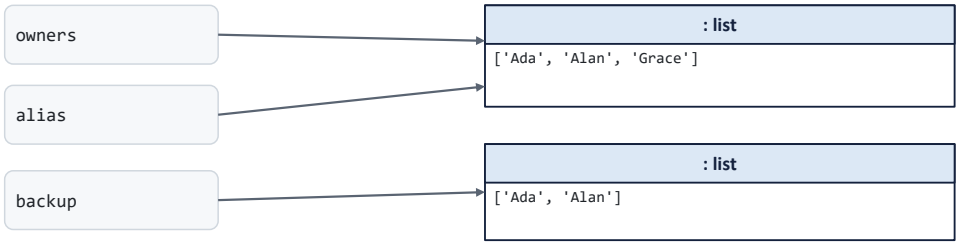
ANSWER

Aliases and Copies

OUTPUT

```
['Ada', 'Alan', 'Grace']
['Ada', 'Alan']
True False
```

- `alias = owners` binds a second name to the **same** list (one object, two names)
- `list(owners)` creates a **new** list: two list objects in total
- Appending through `alias` changes the shared object; `backup` is unaffected



DISCUSS

Find the Objects in a Library System

“Members borrow books. A librarian records each loan with a due date; overdue loans cost a fine per day.”

- Which nouns become **classes**? Which are only **attributes**?
- What is the **state** and **behaviour** of a loan?
- Two copies of the same book: same **identity** or not?
- What would the library system leave out (abstraction)?

02

Defining Classes in Python

Initializers, attributes, methods, properties and special methods

- class, __init__ and self
- Instance vs class attributes
- Properties and information hiding
- Special methods: __repr__, __str__, __eq__, __hash__
- Class/static methods, data classes, type hints

Anatomy of a Class Definition

clock.py

```python
class Time:
    """A time of day on a 24-hour clock."""

    def __init__(self, hour: int, minute: int) -> None:
        self.hour = hour
        self.minute = minute

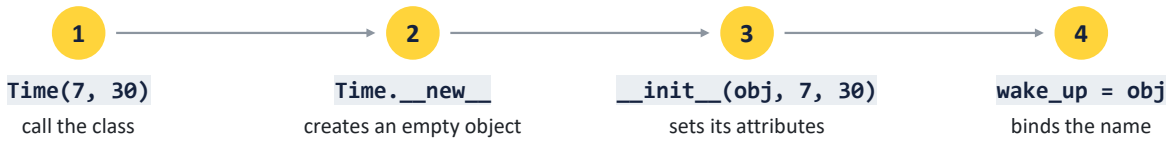
    def to_minutes(self) -> int:
        """Return the minutes elapsed since midnight."""
        return self.hour * 60 + self.minute


wake_up = Time(7, 30)
print(wake_up.hour, wake_up.minute)
print(wake_up.to_minutes())
```

OUTPUT

```
7 30
450
```

Instantiation and `self`




```
>>> class Time:
...     def __init__(self, hour: int, minute: int) -> None:
...         self.hour = hour
...         self.minute = minute
...     def to_minutes(self) -> int:
...         return self.hour * 60 + self.minute
...
>>> wake_up = Time(7, 30)
>>> wake_up.to_minutes()          # method call...
450
>>> Time.to_minutes(wake_up)      # ...is sugar for this
450
>>> vars(wake_up)
{'hour': 7, 'minute': 30}
```


Default Parameter Values

defaults.py

```
class Time:
    def __init__(self, hour: int = 0, minute: int = 0,
                 second: int = 0) -> None:
        self.hour = hour
        self.minute = minute
        self.second = second


for moment in [Time(), Time(9), Time(9, 45), Time(second=30)]:
    print(f"{moment.hour}h:{moment.minute}m:{moment.second}s")
```

OUTPUT

```
0h:0m:0s
9h:0m:0s
9h:45m:0s
0h:0m:30s
```

Watch out

Never use a mutable default such as `tags: list = []` — the one list is shared by every call. Default to `None` and create the list inside `__init__` (details in Module 2).

Instance Attributes vs Class Attributes

attributes.py

```python
class Account:
    interest_rate = 0.03  # class attribute: shared by all
    open_count = 0

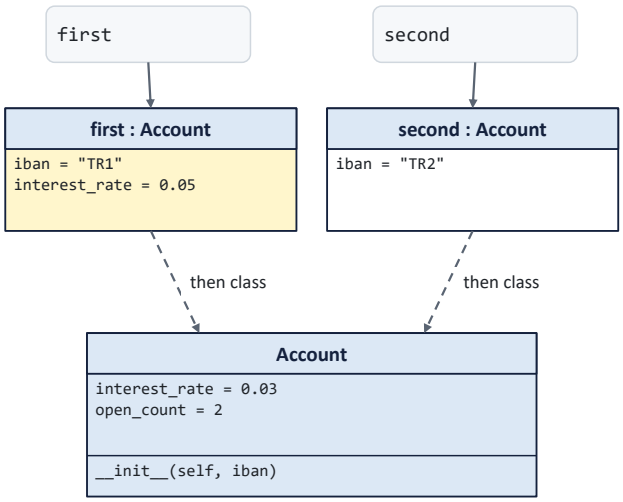
    def __init__(self, iban: str) -> None:
        self.iban = iban  # instance attribute: one per object
        Account.open_count += 1


first = Account("TR1")
second = Account("TR2")
print(first.interest_rate, second.interest_rate, Account.open_count)
first.interest_rate = 0.05  # creates an instance attribute!
print(first.interest_rate, second.interest_rate)
print(Account.interest_rate)
```

OUTPUT

```
0.03 0.03 2
0.05 0.03
0.03
```


Attribute Look-up: Instance First, Then Class




```
>>> class Account:
...     interest_rate = 0.03
...     def __init__(self, iban: str) -> None:
...         self.iban = iban
...
>>> first = Account("TR1")
>>> second = Account("TR2")
>>> first.interest_rate = 0.05
>>> vars(first)
{'iban': 'TR1', 'interest_rate': 0.05}
>>> vars(second)
{'iban': 'TR2'}
>>> second.interest_rate
0.03
```

FIND THE BUG

Why Does Blue Have Ada?

```python
class Team:
    members = []

    def __init__(self, name: str) -> None:
        self.name = name

    def add(self, person: str) -> None:
        self.members.append(person)


red = Team("red")
blue = Team("blue")
red.add("Ada")
print(blue.members)
```

The author expected `[]`. What is printed, and how do you fix the class?

ANSWER

Why Does Blue Have Ada?

```python
class Team:
    def __init__(self, name: str) -> None:
        self.name = name
        self.members: list[str] = []

    def add(self, person: str) -> None:
        self.members.append(person)


red = Team("red")
blue = Team("blue")
red.add("Ada")
print(red.members, blue.members)
```

OUTPUT

```
['Ada'] []
```

- The buggy version prints `['Ada']`
- `members` is a **class** attribute: one list shared by every team
- `self.members.append` reads through the instance and mutates the shared list
- Fix: create the list in `__init__` so each object gets its own

The Account Class: State and Behaviour

account.py

```python
class Account:
    """A bank account identified by its IBAN."""

    def __init__(self, iban: str, balance: float = 100.0) -> None:
        self.iban = iban
        self.balance = balance

    def deposit(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("amount must be positive")
        self.balance += amount

    def withdraw(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("amount must be positive")
        if amount > self.balance:
            raise ValueError("insufficient balance")
        self.balance -= amount
```


Using the Account Class

main.py (continues account.py)

```python
acc = Account("TR1", 100_000.0)
acc.withdraw(2_500.0)
acc.deposit(1_500.0)
print(acc.iban, acc.balance)

for amount in [-10.0, 500_000.0]:
    try:
        acc.withdraw(amount)
    except ValueError as err:
        print(f"withdraw({amount}) failed: {err}")
print(acc.balance)
```

OUTPUT

```
TR1 99000.0
withdraw(-10.0) failed: amount must be positive
withdraw(500000.0) failed: insufficient balance
99000.0
```

Key idea

A failed `withdraw` raises **before** changing anything, so the balance stays valid. Section 3 replaces `ValueError` with a custom `InsufficientBalance` exception.

`__str__` vs `__repr__`

time_repr.py

```python
class Time:
    def __init__(self, hour: int = 0, minute: int = 0) -> None:
        self.hour = hour
        self.minute = minute

    def __repr__(self) -> str:
        return f"Time(hour={self.hour}, minute={self.minute})"

    def __str__(self) -> str:
        return f"{self.hour:02d}:{self.minute:02d}"


lunch = Time(12, 30)
print(lunch)
print(repr(lunch))
print([lunch])
```

OUTPUT

```
12:30
Time(hour=12, minute=30)
[Time(hour=12, minute=30)]
```

Key idea

`repr()` is for developers: unambiguous, ideally valid Python. `str()` is for users: readable. Containers show the `repr` of their items; without `__str__`, `str()` falls back to `__repr__`.

Read-Write Property with Validation

property.py

```python
class Time:
    def __init__(self, hour: int = 0) -> None:
        self.hour = hour  # validated by the setter

    @property  # decorator syntax: see Module 2
    def hour(self) -> int:
        return self._hour

    @hour.setter
    def hour(self, value: int) -> None:
        if not 0 <= value < 24:
            raise ValueError(f"hour must be 0-23, got {value}")
        self._hour = value


wake_up = Time()
wake_up.hour = 10
print(wake_up.hour)
wake_up.hour = 25
```

OUTPUT

```
10
Traceback (most recent call last):
  ...
ValueError: hour must be 0-23, got 25
```

Read-Only Properties

```python
class Time:
    def __init__(self, minute: int = 0) -> None:
        self._minute = minute

    @property
    def minute(self) -> int:
        return self._minute


wake_up = Time(45)
print(wake_up.minute)
wake_up.minute = 20
```

OUTPUT

```
45
Traceback (most recent call last):
  ...
AttributeError: property 'minute' of 'Time' object has no setter
```

Python version note

Python 3.11+ message; older versions printed `AttributeError: can't set attribute`.

Information Hiding: `_name` and `__name`

```
>>> class Account:
...     def __init__(self, balance: float) -> None:
...         self.__balance = balance
...     def report(self) -> str:
...         return f"balance: {self.__balance}"
...
>>> acc = Account(100.0)
>>> acc.report()
'balance: 100.0'
>>> acc.__balance
Traceback (most recent call last):
  ...
AttributeError: 'Account' object has no attribute '__balance'
>>> acc._Account__balance
100.0
```

Name	Meaning
`balance`	public API
`_balance`	internal — by convention only
`__balance`	name-mangled to `_Account__balance`

TRACE THE CODE

What Happened to the Balance?

```
1  class Account:
2      def __init__(self, iban: str, balance: float) -> None:
3          self.iban = iban
4          self.__balance = balance
5
6      def __str__(self) -> str:
7          return f"{self.iban}: {self.__balance}"
8
9
10 acc = Account("TR1", 100.0)
11 acc.__balance = 1_000_000.0
12 print(acc)
13 print(acc.__balance)
14 print(sorted(vars(acc)))
```


ANSWER

What Happened to the Balance?

OUTPUT

```
TR1: 100.0
1000000.0
['_Account__balance', '__balance', 'iban']
```

- Inside the class, `self.__balance` means `self._Account__balance`
- Line 11 is outside the class: no mangling, so it creates a **new** attribute `__balance`
- `__str__` still reads the real, unchanged balance
- Python silently creates attributes on assignment — typos become new attributes

Equality and Hashing: `__eq__` and `__hash__`

```python
class Money:
    def __init__(self, amount: int, currency: str) -> None:
        self.amount = amount
        self.currency = currency

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Money):
            return NotImplemented
        return (self.amount, self.currency) == (other.amount, other.currency)

    def __hash__(self) -> int:
        return hash((self.amount, self.currency))


price, same = Money(10, "EUR"), Money(10, "EUR")
print(price == same, price is same)
print(len({price, same, Money(5, "EUR")}))
```

OUTPUT

```
True False
2
```

Watch out

Defining `__eq__` sets `__hash__ = None` (unhashable) unless you also define `__hash__`. Equal objects must have equal hashes.

QUIZ

Quick Check: Equality and Hashing

A class defines `__eq__` but not `__hash__`. What happens when you add one of its objects to a `set`?

A. It works; the hash is based on `id()`

B. `TypeError: unhashable type`

C. Python derives `__hash__` from the attributes

D. The set falls back to comparing with `is`

ANSWER

Quick Check: Equality and Hashing

A class defines `__eq__` but not `__hash__`. What happens when you add one of its objects to a `set`?

A. It works; the hash is based on `id()`

B. `TypeError: unhashable type`

C. Python derives `__hash__` from the attributes

D. The set falls back to comparing with `is`

Why

Overriding `__eq__` without `__hash__` sets `__hash__ = None`, so the objects are unhashable and cannot be set members or dictionary keys.

2 · DEFINING CLASSES IN PYTHON

`@classmethod` and `@staticmethod`

factories.py

```python
from typing import Self


class Time:
    def __init__(self, hour: int = 0, minute: int = 0) -> None:
        self.hour = hour
        self.minute = minute

    @classmethod
    def from_string(cls, text: str) -> Self:
        hour, minute = text.split(":")
        return cls(int(hour), int(minute))

    @staticmethod
    def is_valid(hour: int, minute: int) -> bool:
        return 0 <= hour < 24 and 0 <= minute < 60


meeting = Time.from_string("14:30")
print(meeting.hour, meeting.minute, Time.is_valid(25, 0))
```

OUTPUT

```
14 30 False
```

Three Kinds of Methods

	Instance method	`@classmethod`	`@staticmethod`
First parameter	`self` (the object)	`cls` (the class)	none
Can use	object and class state	class state	only its arguments
Typical use	behaviour of one object	alternative constructors, registries	helpers that belong to the class
Called as	`obj.method()`	`Cls.method()`	`Cls.method()`

Tip

If a static method has nothing to do with the class, a module-level function is simpler and just as Pythonic.

Data Classes: Less Boilerplate

dataclass_time.py

```python
from dataclasses import dataclass


@dataclass(order=True)
class Time:
    hour: int = 0
    minute: int = 0
    second: int = 0


start = Time(9, 15)
print(start)
print(start == Time(9, 15, 0))
print(Time(minute=5) < start)
```

OUTPUT

```
Time(hour=9, minute=15, second=0)
True
True
```

Generated	Based on
`__init__`	the annotated fields, in order
`__repr__`	`Time(hour=9, minute=15, second=0)`
`__eq__`	field-by-field comparison
`__lt__`, `__le__`, …	`order=True`: compares as tuples

Tip

A data class is still a normal class: add methods and properties as usual.

Data Classes: `frozen`, `slots` and `field`

wallet.py

```python
from dataclasses import dataclass, field


@dataclass(frozen=True, slots=True)
class Money:
    amount: int
    currency: str = "EUR"


@dataclass
class Wallet:
    owner: str
    coins: list[Money] = field(default_factory=list)


wallet = Wallet("Ada")
wallet.coins.append(Money(5))
print(wallet.coins, len({Money(5), Money(5, "EUR")}))
```

OUTPUT

```
[Money(amount=5, currency='EUR')] 1
```

Option	Effect
`frozen=True`	immutable fields; hashable → usable in sets and as keys
`slots=True`	uses `__slots__`: less memory, no accidental new attributes (3.10+)
`field(default_factory=list)`	a fresh list per object — no shared mutable default
`kw_only=True`	keyword-only `__init__` parameters (3.10+)

Type Hints: Annotating Your Code

hints.py

```python
def average(values: list[float]) -> float:
    return sum(values) / len(values)


def find_rate(iban: str,
              rates: dict[str, float]) -> float | None:
    return rates.get(iban)


rates: dict[str, float] = {"TR1": 0.03}
print(average([1.0, 2.0, 4.5]))
print(find_rate("TR1", rates), find_rate("TR9", rates))
print(average.__annotations__)
```

OUTPUT

```
2.5
0.03 None
{'values': list[float], 'return': <class 'float'>}
```

Key idea

Hints are **not enforced** at runtime. Static checkers such as **mypy** and your IDE use them to find type errors before the code runs.

Type Hints: Common Forms

Hint	Meaning
`int`, `float`, `str`, `bool`	built-in types
`list[str]`, `set[int]`	collections of one element type (3.9+)
`dict[str, float]`	mapping from keys to values
`tuple[int, int]`, `tuple[int, ...]`	fixed-length / variable-length tuples
`X | None`	optional value (3.10+; formerly `Optional[X]`)
`Self` (from `typing`)	an instance of the current class (3.11+)

TERMINAL

```
$ mypy bank.py
bank.py:14: error: Argument 1 to "deposit" of "Account" has incompatible type "str"; expected "float"  [arg-type]
Found 1 error in 1 file (checked 1 source file)
```


TRY IT

A Temperature Class

Write a class `Temperature` that:

- stores degrees Celsius in a read-write property `celsius`
  - the setter rejects values below −273.15 with `ValueError`
- offers a read-only property `fahrenheit` (°F = °C × 9/5 + 32)
- has a `__repr__` such as `Temperature(celsius=21.5)`
- *Bonus:* an alternative constructor `Temperature.from_fahrenheit(212)`

```
>>> room = Temperature(21.5)
>>> room, room.fahrenheit
(Temperature(celsius=21.5), 70.7)
```

ANSWER

A Temperature Class

```python
class Temperature:
    def __init__(self, celsius: float) -> None:
        self.celsius = celsius

    @property
    def celsius(self) -> float:
        return self._celsius

    @celsius.setter
    def celsius(self, value: float) -> None:
        if value < -273.15:
            raise ValueError(f"below absolute zero: {value}")
        self._celsius = value

    @property
    def fahrenheit(self) -> float:
        return self._celsius * 9 / 5 + 32

    def __repr__(self) -> str:
        return f"Temperature(celsius={self._celsius})"


room = Temperature(21.5)
print(room, room.fahrenheit)
```

OUTPUT

```
Temperature(celsius=21.5) 70.7
```


03

Exceptions and Custom Exception Classes

Raising, handling, propagating and designing exceptions

- How exceptions propagate
- try / except / else / finally
- The built-in exception hierarchy
- Custom exceptions and chaining
- EAFP vs LBYL

How an Exception Propagates

propagate.py

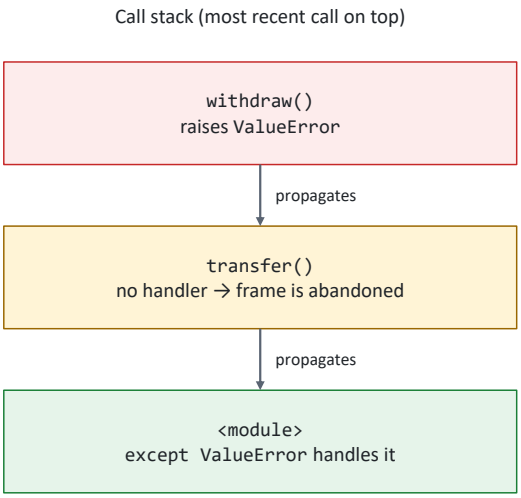
```python
def withdraw(balance: float, amount: float) -> float:
    if amount > balance:
        raise ValueError("insufficient balance")
    return balance - amount


def transfer(balance: float, amount: float) -> float:
    return withdraw(balance, amount)  # no handler


try:
    transfer(50.0, 80.0)
except ValueError as err:
    print(f"transfer failed: {err}")
```

OUTPUT

```
transfer failed: insufficient balance
```



`try`, `except`, `else`, `finally`

clauses.py

```python
def parse_amount(text: str) -> float | None:
    try:
        amount = float(text)
    except ValueError:
        print(f"not a number: {text!r}")
        return None
    else:
        print(f"parsed {amount}")
        return amount
    finally:
        print("finally runs in every case")


parse_amount("12.5")
parse_amount("twelve")
```

OUTPUT

```
parsed 12.5
finally runs in every case
not a number: 'twelve'
finally runs in every case
```

Clause	Runs when…
`try`	always: the code that may fail
`except E`	an exception matching `E` was raised in `try`
`else`	`try` finished **without** an exception
`finally`	always — even after `return` or an unhandled exception

Tip

Keep `try` blocks small; put the code that should run only on success in `else`.

Handling Several Exception Types

ratio.py

```python
def safe_ratio(a: str, b: str) -> float | None:
    try:
        return float(a) / float(b)
    except ZeroDivisionError:
        print("cannot divide by zero")
    except (TypeError, ValueError) as err:
        print(f"bad input: {err}")
    return None


print(safe_ratio("3", "4"))
print(safe_ratio("3", "0"))
print(safe_ratio("3", "x"))
```

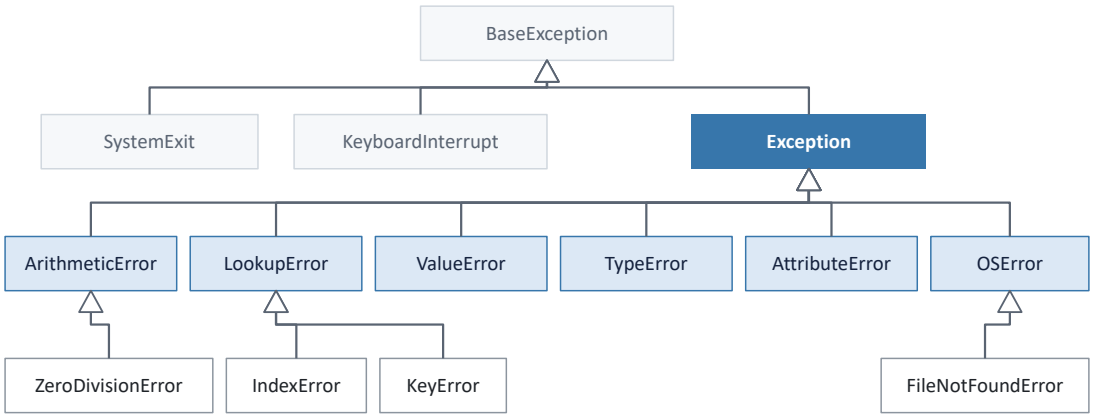
OUTPUT

```
0.75
cannot divide by zero
None
bad input: could not convert string to float: 'x'
None
```

Watch out

Python runs the **first** matching `except`, so list specific types before general ones. Never write a bare `except:` — it even catches `KeyboardInterrupt` (Ctrl+C).

The Built-in Exception Hierarchy



Key idea

`except LookupError` catches both `IndexError` and `KeyError`. Catch `Exception` or narrower — never `BaseException`, which also stops Ctrl+C and `sys.exit()`.

Raising and Re-raising Exceptions

raising.py

```python
def load_rate(text: str) -> float:
    try:
        rate = float(text)
    except ValueError:
        print(f"log: bad rate {text!r}")
        raise  # re-raise the same exception
    if not 0 <= rate <= 1:
        raise ValueError(f"rate {rate} not in [0, 1]")
    return rate


for text in ["0.25", "1.5", "abc"]:
    try:
        print(load_rate(text))
    except ValueError as err:
        print("rejected:", err)
```

OUTPUT

```
0.25
rejected: rate 1.5 not in [0, 1]
log: bad rate 'abc'
rejected: could not convert string to float: 'abc'
```

Watch out

Use `assert` only for internal invariants, never to validate input: `python -O` removes all `assert` statements.

A Custom Exception: `InsufficientBalance`

errors.py

```python
class InsufficientBalance(Exception):
    """Raised when a withdrawal exceeds the balance."""

    def __init__(self, balance: float, amount: float) -> None:
        super().__init__(f"balance {balance} is less than {amount}")
        self.balance = balance
        self.amount = amount

    @property
    def deficit(self) -> float:
        return self.amount - self.balance


try:
    raise InsufficientBalance(97_500.0, 100_000.0)
except InsufficientBalance as err:
    print(err)
    print(f"deficit: {err.deficit}")
```

OUTPUT

```
balance 97500.0 is less than 100000.0
deficit: 2500.0
```

A Custom Exception Hierarchy

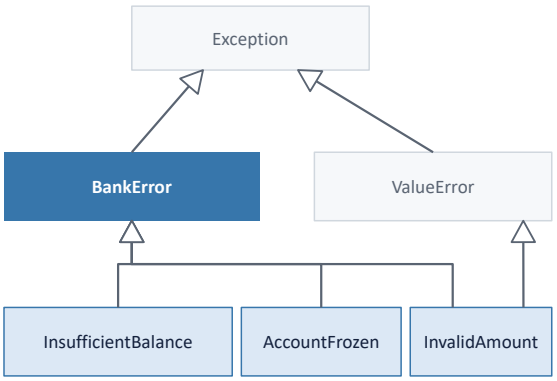
errors.py

```python
class BankError(Exception):
    """Root of all errors raised by the bank."""


class InvalidAmount(BankError, ValueError):
    """Amount is zero or negative."""


class InsufficientBalance(BankError):
    ...  # __init__ and deficit as before


class AccountFrozen(BankError):
    """Account is locked by compliance."""
```



Tip

Callers catch one family with `except BankError`; `InvalidAmount` still works with `except ValueError`.

Handling Errors by Category

account.py

```python
def withdraw(self, amount: float) -> None:  # method of Account
    if amount <= 0:
        raise InvalidAmount(f"amount must be positive: {amount}")
    if amount > self.balance:
        raise InsufficientBalance(self.balance, amount)
    self.balance -= amount
```

main.py

```python
acc = Account("TR1", 100.0)
for amount in [30.0, -5.0, 500.0]:
    try:
        acc.withdraw(amount)
    except InsufficientBalance as err:
        print(f"declined: short by {err.deficit}")
    except BankError as err:
        print(f"bank error: {err}")
print(acc.balance)
```

OUTPUT

```
bank error: amount must be positive: -5.0
declined: short by 430.0
70.0
```

Exception Chaining with `raise ... from`

chaining.py

```python
class ConfigError(Exception):
    """Configuration could not be loaded."""


def read_port(settings: dict[str, str]) -> int:
    try:
        return int(settings["port"])
    except (KeyError, ValueError) as err:
        raise ConfigError("invalid or missing 'port'") from err


try:
    read_port({"port": "eighty"})
except ConfigError as err:
    print(err)
    print("caused by:", repr(err.__cause__))
```

OUTPUT

```
invalid or missing 'port'
caused by: ValueError("invalid literal for int() with base 10: 'eighty'")
```


EAFP vs LBYL

LBYL — *look before you leap*

```python
if "port" in settings:
    raw = settings["port"]
    port = int(raw) if raw.isdigit() else 8080
else:
    port = 8080
```

EAFP — *easier to ask forgiveness than permission*

```python
try:
    port = int(settings["port"])
except (KeyError, ValueError):
    port = 8080
```

	LBYL	EAFP
Idea	check every precondition first	just do it; handle the exception
Race conditions	state may change between check and use	none: one atomic attempt
Typical Python use	cheap, common checks (`if key in d`)	preferred default style

TRACE THE CODE

Follow the Clauses

```python
def check(n: int) -> str:
    try:
        if n < 0:
            raise ValueError("negative")
        result = "ok"
    except ValueError as err:
        return f"error: {err}"
    else:
        result += "!"
    finally:
        print("finally", n)
    return result


print(check(1))
print(check(-1))
```

Predict all four output lines — in the right order.

ANSWER

Follow the Clauses

OUTPUT

```
finally 1
ok!
finally -1
error: negative
```

- `finally` runs **before** the function returns, so its line comes first
- `check(1)`: no exception → `else` appends `!` → `finally` → `return "ok!"`
- `check(-1)`: `except` prepares the return value, `finally` prints, then the function returns

QUIZ

Quick Check: Exception Hierarchy

Which single clause handles both a missing dictionary key and a list index out of range — and nothing unrelated?

A `except KeyError:`

B `except LookupError:`

C `except ValueError:`

D `except BaseException:`

ANSWER

Quick Check: Exception Hierarchy

Which single clause handles both a missing dictionary key and a list index out of range — and nothing unrelated?

A `except KeyError:`

B `except LookupError:`

C `except ValueError:`

D `except BaseException:`

★ **Why**

`KeyError` and `IndexError` both inherit from `LookupError`. `BaseException` would catch them too, but also `SystemExit`, `KeyboardInterrupt` and every bug.

TRY IT

Add a Withdrawal Limit

- Define `LimitExceeded(BankError)` storing `limit` and `amount` as attributes
- Give `Account` a `daily_limit` (default `1000.0`) and raise `LimitExceeded` in `withdraw` when `amount > daily_limit`
- The caller prints `limit 1000.0 exceeded by 250.0` for `withdraw(1250.0)`
- *Bonus:* `read_limit(text)` that turns a `ValueError` from `float(text)` into a `BankError` with `raise ... from`

```
>>> acc = Account("TR1", 5000.0)
>>> acc.withdraw(1250.0)
Traceback (most recent call last):
  ...
LimitExceeded: limit 1000.0 exceeded by 250.0
```

✓ **Hint**

Call `super().__init__(message)` in `LimitExceeded.__init__`.

04

Inheritance and Polymorphism

Reusing and specialising classes — and treating different objects alike

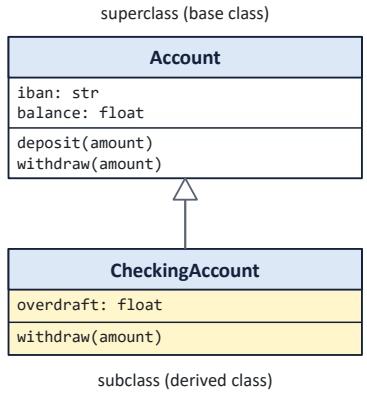
- Inheritance, super() and overriding
- is-a tests: isinstance and issubclass
- Multiple inheritance and the MRO
- Polymorphism and duck typing
- Operator overloading, ABCs and Protocols

Inheritance

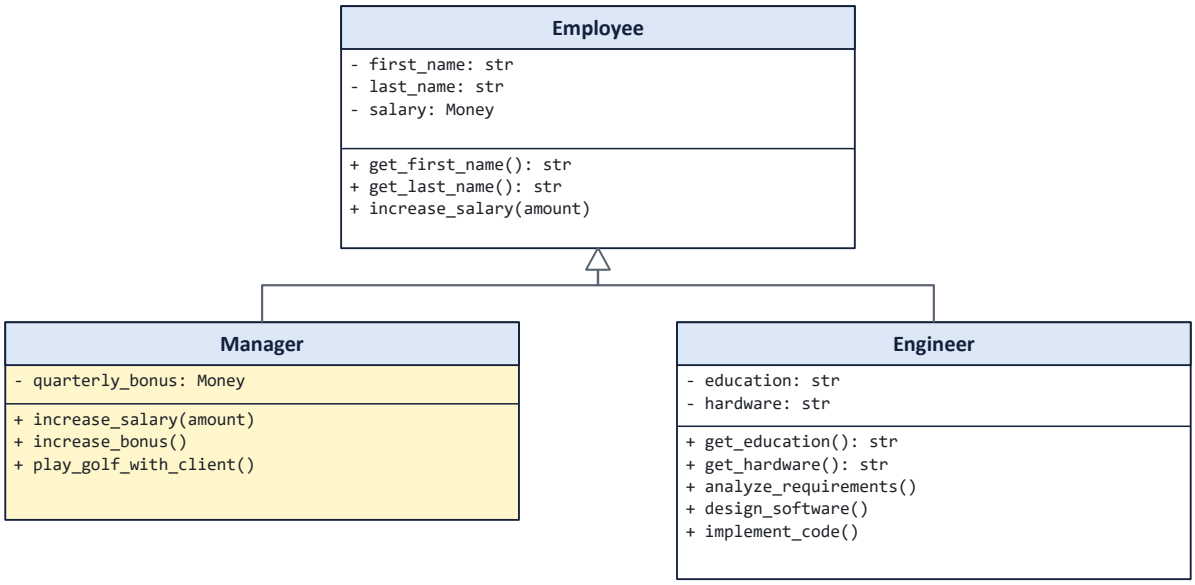
Definition

"A mechanism whereby a class is defined in reference to others, adding all their features to its own." — Meyer, *Object-Oriented Software Construction*

- The **subclass** inherits the attributes and methods of its **superclass**
- It can **add** members and **override** inherited methods
- Plausible only if a subclass object *is a kind of* superclass object
- …and substitutable for it: the **Liskov Substitution Principle** (Section 7)
- Python allows **multiple** inheritance; Java only single class inheritance



Inheritance in UML: The Employee Hierarchy



Subclassing: `CheckingAccount`

checking.py

```python
class CheckingAccount(Account):
    """An account that may be overdrawn up to a limit."""

    def __init__(self, iban: str, balance: float = 0.0,
                 overdraft: float = 0.0) -> None:
        super().__init__(iban, balance)
        self.overdraft = overdraft

    def withdraw(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("amount must be positive")
        if amount > self.balance + self.overdraft:
            raise ValueError("overdraft limit exceeded")
        self.balance -= amount
```

Key idea

`super().__init__(...)` runs the parent's initializer — Python does **not** call it automatically. Forget it and `iban` and `balance` are never set.

Inherited, Overridden and Added Members

main.py

```python
checking = CheckingAccount("TR2", 1000.0, overdraft=500.0)
checking.withdraw(1250.0)  # overridden in CheckingAccount
checking.deposit(100.0)    # inherited from Account
print(checking.balance, checking.overdraft)
print([cls.__name__ for cls in CheckingAccount.__mro__])
```

OUTPUT

```
-150.0 500.0
['CheckingAccount', 'Account', 'object']
```

Member	Found in	How
`withdraw`	`CheckingAccount`	overridden: replaces the inherited version
`deposit`	`Account`	inherited unchanged
`overdraft`	the object itself	added by the subclass: set in `CheckingAccount.__init__`
`iban`, `balance`	the object itself	set by `Account.__init__`, called via `super()`

Testing "Is-A": `isinstance` and `issubclass`

```
>>> class Account: ...
>>> class CheckingAccount(Account): ...
>>> savings = Account()
>>> checking = CheckingAccount()
>>> isinstance(checking, Account)
True
>>> isinstance(savings, CheckingAccount)
False
>>> issubclass(CheckingAccount, Account)
True
>>> isinstance(checking, (int, Account))
True
>>> type(checking) is Account
False
```

Call	Question
`isinstance(obj, C)`	is `obj` a `C` or an instance of a subclass of `C`?
`issubclass(D, C)`	is class `D` derived from `C` (or `C` itself)?
`type(obj) is C`	is the class of `obj` exactly `C`?

Tip

Prefer `isinstance` over comparing types — and prefer polymorphism over both.

Multiple Inheritance and Mixins

mixins.py

```python
class ReprMixin:
    def __repr__(self) -> str:
        fields = ", ".join(f"{k}={v!r}" for k, v in vars(self).items())
        return f"{type(self).__name__}({fields})"


class AuditMixin:
    def audit(self, action: str) -> str:
        return f"AUDIT {type(self).__name__}: {action}"


class SavingsAccount(ReprMixin, AuditMixin):
    def __init__(self, iban: str, rate: float) -> None:
        self.iban = iban
        self.rate = rate


savings = SavingsAccount("TR3", 0.04)
print(savings)
print(savings.audit("interest added"))
```

OUTPUT

```
SavingsAccount(iban='TR3', rate=0.04)
AUDIT SavingsAccount: interest added
```

The Diamond Problem

diamond.py

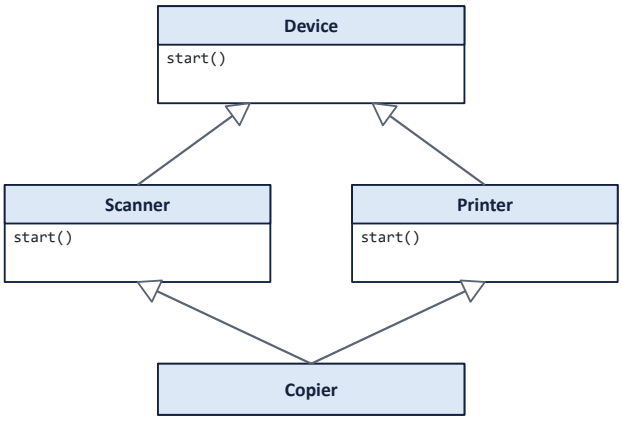
```python
class Device:
    def start(self) -> str:
        return "Device"


class Scanner(Device):
    def start(self) -> str:
        return "Scanner > " + super().start()


class Printer(Device):
    def start(self) -> str:
        return "Printer > " + super().start()


class Copier(Scanner, Printer):
    pass


print(Copier().start())
print([c.__name__ for c in Copier.__mro__])
```



OUTPUT

```
Scanner > Printer > Device
['Copier', 'Scanner', 'Printer', 'Device', 'object']
```


Method Resolution Order (MRO)



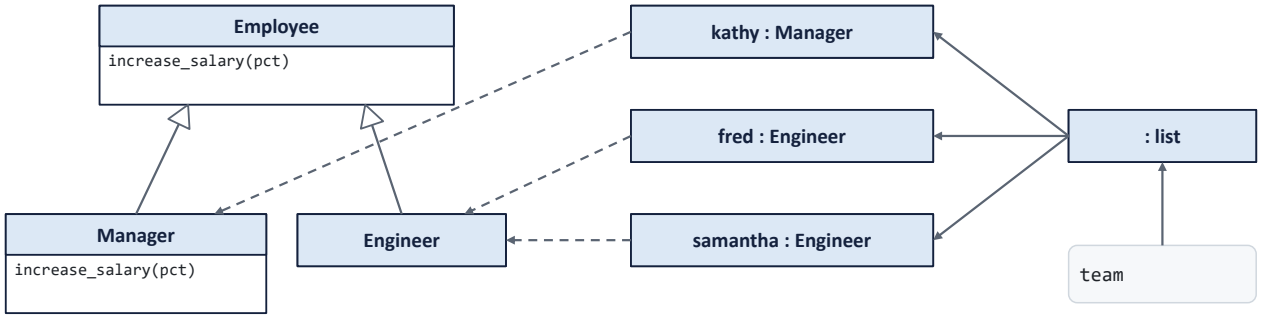
- Python linearizes the class graph with the **C3** algorithm
- Subclasses come before their bases; the order of bases is preserved
- Every class appears exactly once, so `Device.start` runs once
- `super()` means *the next class in the object's MRO* — not “my parent”
- Inspect it with `Copier.__mro__` or `Copier.mro()`

> **Watch out**
>
> If no consistent order exists — e.g. `class Bad(Device, Scanner)` — Python raises `TypeError` when the class statement runs.

Polymorphism

Definition

"A name may denote objects of many different classes that are related by some common superclass." — Booch, *Object-Oriented Analysis and Design*




```python
for employee in team:              # one name, several classes
    employee.increase_salary(10)   # the object's class picks the method
```


Polymorphism in Python

```python
class Employee:
    def __init__(self, name: str, salary: float) -> None:
        self.name = name
        self.salary = salary

    def increase_salary(self, percent: float) -> None:
        self.salary *= 1 + percent / 100


class Manager(Employee):
    def increase_salary(self, percent: float) -> None:
        super().increase_salary(percent + 5)  # +5% bonus


team = [Manager("Kathy", 5000), Employee("Fred", 4000)]
for member in team:
    member.increase_salary(10)
    print(member.name, round(member.salary))
```

OUTPUT

```
Kathy 5750
Fred 4400
```

Key idea

No type checks in the loop: a new subclass needs **no change** here (Open/Closed, Section 6).

Duck Typing

```python
import io
import sys


class UpperWriter:
    def write(self, text: str) -> int:
        return sys.stdout.write(text.upper())


def report(out) -> None:  # any object with .write(str)
    out.write("balance: 42\n")


buffer = io.StringIO()
for target in [sys.stdout, UpperWriter(), buffer]:
    report(target)
print(repr(buffer.getvalue()))
```

OUTPUT

```
balance: 42
BALANCE: 42
'balance: 42\n'
```

Duck typing

"If it walks like a duck and quacks like a duck, it is a duck."

Operator Overloading

Operators are syntax for special methods: `a + b` calls `a.__add__(b)`.

Expression	Special method
`a + b`, `a - b`	`__add__`, `__sub__`
`a * b`, `a / b`, `a // b`	`__mul__`, `__truediv__`, `__floordiv__`
`a == b`, `a < b`	`__eq__`, `__lt__`
`-a`, `abs(a)`	`__neg__`, `__abs__`
`len(a)`, `a[i]`	`__len__`, `__getitem__`
`2 + a` (reflected)	`__radd__` on `a`

Restrictions

- Precedence and associativity stay the same
- Arity stays the same: binary stays binary
- No new operators — only existing ones
- Built-in types keep their meaning: `1 + 2` is always `3`

Fraction: Construction and Normalization

fraction.py

```python
import math


class Fraction:
    def __init__(self, numerator: int, denominator: int = 1) -> None:
        if denominator == 0:
            raise ZeroDivisionError("denominator must not be zero")
        if denominator < 0:
            numerator, denominator = -numerator, -denominator
        common = math.gcd(numerator, denominator)
        self.numerator = numerator // common
        self.denominator = denominator // common

    def __repr__(self) -> str:
        return f"Fraction({self.numerator}, {self.denominator})"

    def __str__(self) -> str:
        return f"{self.numerator}/{self.denominator}"
```


Fraction: Arithmetic Operators

fraction.py (continued)

```python
class Fraction:  # continued
    def __add__(self, other: "Fraction") -> "Fraction":
        return Fraction(self.numerator * other.denominator
                        + other.numerator * self.denominator,
                        self.denominator * other.denominator)

    def __sub__(self, other: "Fraction") -> "Fraction":
        return self + Fraction(-other.numerator, other.denominator)

    def __mul__(self, other: "Fraction") -> "Fraction":
        return Fraction(self.numerator * other.numerator,
                        self.denominator * other.denominator)

    def __truediv__(self, other: "Fraction") -> "Fraction":
        return self * Fraction(other.denominator, other.numerator)

    def __floordiv__(self, other: "Fraction") -> int:
        quotient = self / other
        return quotient.numerator // quotient.denominator
```

Fraction in Action

main.py

```
half = Fraction(1, 2)
three_quarters = Fraction(3, 4)
print(half + three_quarters)
print(half - three_quarters)
print(half * three_quarters)
print(half / three_quarters)
print(half // three_quarters, Fraction(7, 2) // half)
print(repr(Fraction(6, -8)))
```

OUTPUT

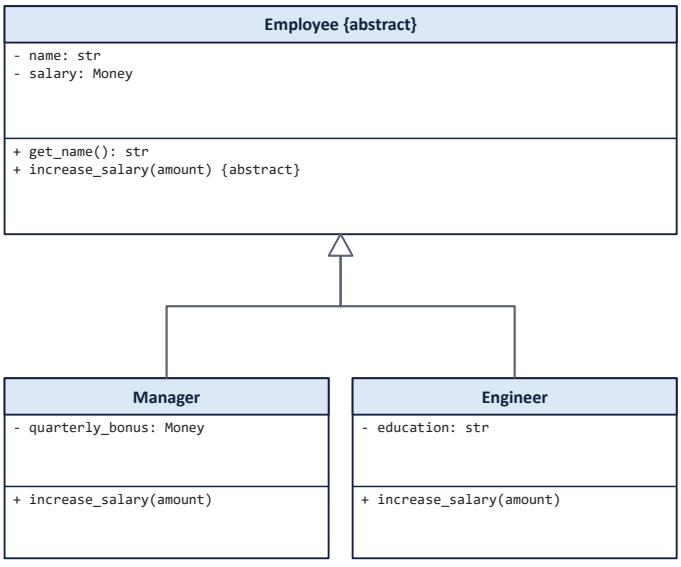
```
5/4
-1/4
3/8
2/3
0 7
Fraction(-3, 4)
```

Tip

In real code use the standard library: `fractions.Fraction` supports all operators, comparisons, hashing and mixing with `int`.

Abstract Classes

- An **abstract class** cannot be instantiated
- It may have attributes, concrete methods and an initializer
- **Abstract methods** declare an operation without implementing it
- Concrete subclasses must implement every abstract method
- UML: name in *italics* or marked `{abstract}`



Abstract Base Classes in Python

```python
from abc import ABC, abstractmethod


class Employee(ABC):
    def __init__(self, salary: float) -> None:
        self.salary = salary

    @abstractmethod
    def increase_salary(self, percent: float) -> None:
        """Concrete subclasses must implement this."""

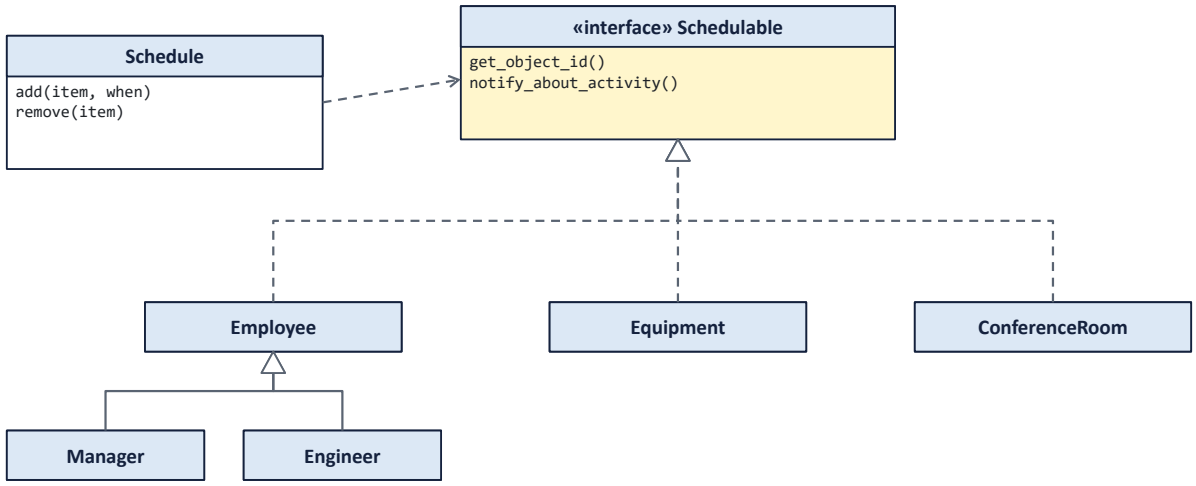

class Engineer(Employee):
    def increase_salary(self, percent: float) -> None:
        self.salary *= 1 + percent / 100


print(Engineer(4000).salary)
Employee(5000)
```

OUTPUT

```
4000
TypeError: Can't instantiate abstract class Employee without an implementation for abstract method 'increase_salary'
```


Interfaces



An **interface** declares operations only. Unrelated classes *implement* it (dashed arrow), and clients such as `Schedule` depend only on the interface.

Structural Interfaces with `typing.Protocol`

```python
from typing import Protocol


class Schedulable(Protocol):
    def notify(self, activity: str) -> str: ...


class Engineer:
    def notify(self, activity: str) -> str:
        return f"engineer invited to {activity}"


class Projector:
    def notify(self, activity: str) -> str:
        return f"projector booked for {activity}"


items: list[Schedulable] = [Engineer(), Projector()]
for item in items:
    print(item.notify("design review"))
```

OUTPUT

```
engineer invited to design review
projector booked for design review
```



No inheritance needed: a matching `notify` method is enough (*structural typing*).

ABC or Protocol?

	Java interface	`abc.ABC`	`typing.Protocol`
Relationship	declared: `implements`	nominal: must inherit	structural: matching methods
Attributes	constants only	allowed	declared by annotations
Implementation	abstract (+ default methods)	abstract and concrete methods	usually none
Missing method caught	by the compiler	`TypeError` on instantiation	by mypy / IDE
`isinstance` check	yes	yes	only with `@runtime_checkable`



Key idea

Use an **ABC** to share implementation in a class family you own. Use a **Protocol** to describe what *your code needs* from objects you do not control.

TRACE THE CODE

Who Answers First?

```python
class Base:
    def greet(self) -> str:
        return "Base"


class Left(Base):
    def greet(self) -> str:
        return "Left > " + super().greet()


class Right(Base):
    def greet(self) -> str:
        return "Right > " + super().greet()


class Child(Right, Left):
    pass


print(Child().greet())
print(Left().greet())
```


ANSWER

Who Answers First?

OUTPUT

```
Right > Left > Base
Left > Base
```

- MRO of `Child`: `Child`, `Right`, `Left`, `Base`, `object`
- In a `Child` object, `super()` inside `Right` continues with `Left` — not `Base`
- In a plain `Left` object, the next class after `Left` is `Base`
- `super()` depends on the **object's** MRO, not on where the method is written

QUIZ

Quick Check: Protocols

`Projector` does not inherit from the protocol `Schedulable`. When does mypy accept a `Projector` where a `Schedulable` is expected?

- **A** Never — it must inherit from `Schedulable`
- **B** When it has a matching `notify` method
- **C** Only after `Schedulable.register(Projector)`
- **D** Only if the function checks `isinstance` first

ANSWER

Quick Check: Protocols

`Projector` does not inherit from the protocol `Schedulable`. When does mypy accept a `Projector` where a `Schedulable` is expected?

- **A** Never — it must inherit from `Schedulable`
- **B** When it has a matching `notify` method
- **C** Only after `Schedulable.register(Projector)`
- **D** Only if the function checks `isinstance` first

★ **Why**

Protocols are structural: any class whose methods match the protocol's signatures satisfies it. Registration belongs to ABCs; `isinstance` is irrelevant to static checking.

TRY IT

Complete the Fraction Class

- Add `__eq__` so that `Fraction(1, 2) == Fraction(2, 4)` is `True`
- Add `__lt__` so that `sorted()` works on a list of fractions
- Add `__hash__` consistent with `__eq__`
- Add `__radd__` so that `1 + Fraction(1, 2)` gives `3/2`
- *Bonus:* accept `int` operands in `__add__` (convert with `Fraction(other)`)

Hint

Return `NotImplemented` for operand types you do not support.

05

Object Design Fundamentals

Complexity, cohesion, coupling, composition and delegation

- Complexity, decomposition and costs
- Cohesion and coupling
- Implementation vs interface inheritance
- Composition, associations and delegation
- Favour composition; program to an interface

Why Object-Oriented Design?

Definition

"Software systems perform certain actions on objects of certain types; to obtain flexible and reusable systems, it is better to base their structure on the object types than on the actions." — Meyer, *Object-Oriented Software Construction*

Complexity

Objects mirror the hierarchical structure of complex systems

Decomposition

Split a system into interacting objects, not a tree of procedures

Development cost

Business entities are modelled early and map directly onto code

Maintenance cost

OO models adapt to new requirements, mostly by adding classes

Key idea

Most of a system's lifetime cost is spent *after* the first release — design for change.

Software Complexity

1. Hierarchy

Systems are made of interrelated subsystems, recursively

2. Relative primitives

Which components count as primitive is the observer's choice

3. Separation of concerns

Links inside a component are stronger than links between components, so parts can be studied in isolation

4. Common patterns

Few kinds of components, combined in many ways

5. Stable intermediate forms

A working complex system evolves from a working simple one

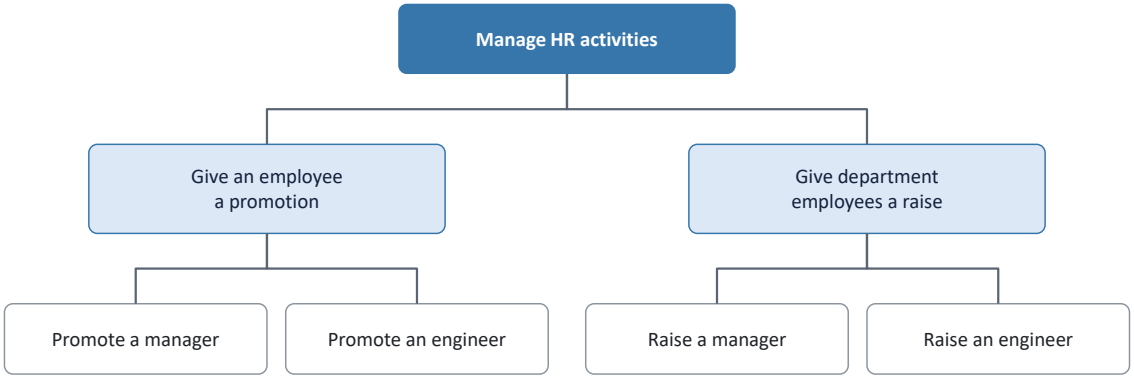
6. In Python

package → module → class → method: the same hierarchy at every level

After Booch, Object-Oriented Analysis and Design — the fifth point is also known as Gall's law.

Procedural Decomposition

The procedural paradigm decomposes software into a hierarchy of **procedures** (tasks).

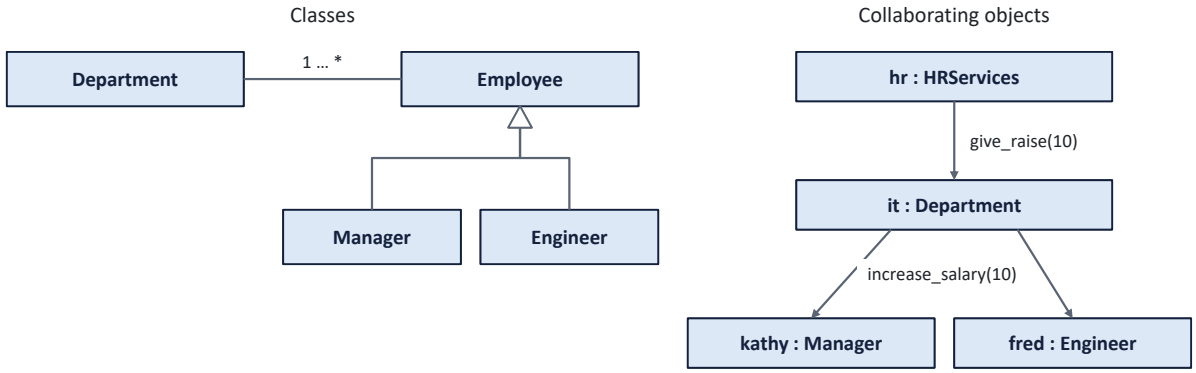


Watch out

Every task knows every kind of employee: adding a `Director` means editing many procedures.

Object-Oriented Decomposition

The OO paradigm decomposes software into a hierarchy of **interacting objects**.



Key idea

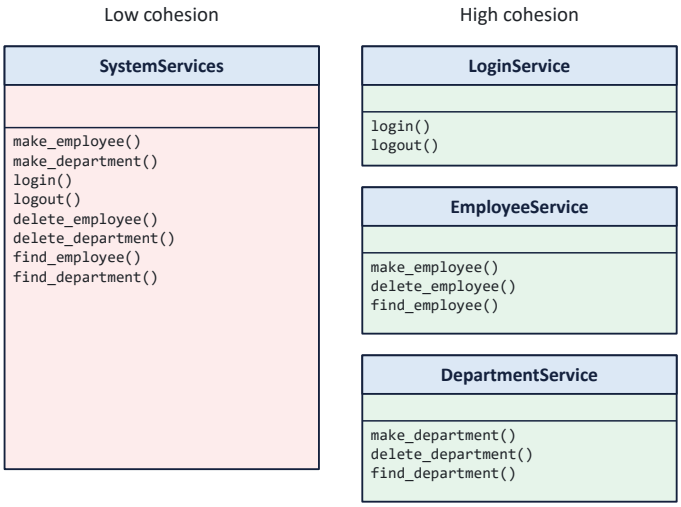
Each object does its own part: the department forwards the request, each employee applies its own rule.

Cohesion

- **Cohesion**: how well a component supports a *single* purpose
- Low: many unrelated responsibilities — the *Blob* anti-pattern
- High: one set of related responsibilities
- Applies to methods, classes, modules, packages and systems

Tip

Aim for **high cohesion**: each class should be easy to name and describe in one sentence.



Coupling

Definition

"The degree to which classes within our system are dependent on each other." — Knoernschild, *Java Design*



Implementation Inheritance

Inheriting shared attributes **and implementation** from a superclass.

Advantages	Disadvantages
Avoids duplicating code common to subtypes	The subclass inherits *everything* — wanted or not
Organizes classes in a hierarchy	Superclass changes ripple into every subclass

white_box.py

```python
class DepartmentEmployees(list):
    def __init__(self, department: str) -> None:
        super().__init__()
        self.department = department

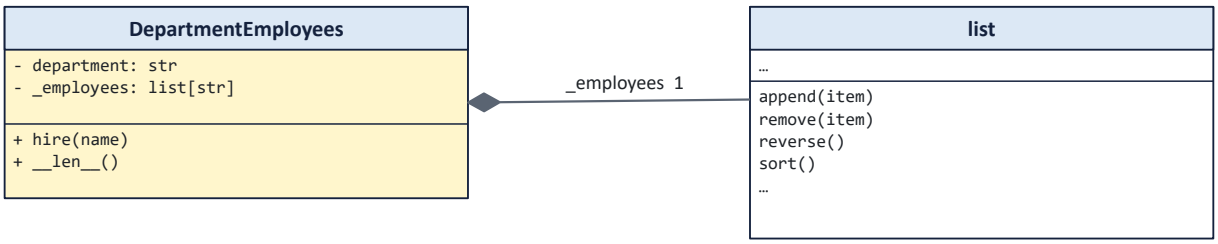

staff = DepartmentEmployees("IT")
staff.append("Fred")
staff.append(42)  # accepted!
staff.reverse()   # list API leaks
print(staff, staff.department)
```

OUTPUT

```
[42, 'Fred'] IT
```


Composition

- Build complex objects from simpler objects: a class **has a** part
- Looser coupling than implementation inheritance (*black-box* reuse)
- The whole controls which operations of the part it exposes



> **Note**
>
> UML: a filled diamond marks **composition** (the part lives and dies with the whole); a hollow diamond marks the weaker **aggregation**.

Composition in Python

black_box.py

```python
class DepartmentEmployees:
    """A department's staff list, built by composition."""

    def __init__(self, department: str) -> None:
        self.department = department
        self._employees: list[str] = []  # has-a list

    def hire(self, name: str) -> None:
        if name in self._employees:
            raise ValueError(f"{name} already works here")
        self._employees.append(name)

    def __len__(self) -> int:
        return len(self._employees)


it = DepartmentEmployees("IT")
it.hire("Fred")
it.hire("Samantha")
print(it.department, len(it), hasattr(it, "reverse"))
```

OUTPUT

```
IT 2 False
```


Interface Inheritance

- Separates an interface *definition* from its implementations
- Like hardware: any device with the right port fits the socket
- New implementations extend a system without modifying it
- Java uses interfaces; Python uses ABCs with only abstract methods, or Protocols

schedulable.py

```python
from abc import ABC, abstractmethod


class Schedulable(ABC):
    @abstractmethod
    def cancel_activity(self) -> None: ...

    @abstractmethod
    def notify_about_activity(self) -> None: ...

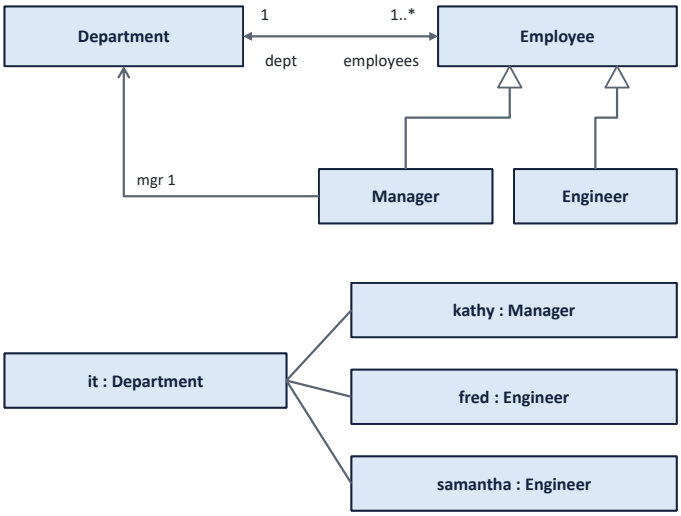

class Equipment(Schedulable):
    def cancel_activity(self) -> None:
        print("Equipment: activity cancelled")

    def notify_about_activity(self) -> None:
        print("Equipment: notified")
```

Associations: Roles, Multiplicity and Links

Multiplicity	Meaning
`1`	exactly one
`0..1`	zero or one
`1..*`	one or more
`0..*` or `*`	zero or more

- **Role**: the name an end plays (`dept`, `employees`, `mgr`)
- **Navigability**: which end can reach the other (arrowhead)
- **Link**: an instance of an association between objects



Associations in Python Code

association.py

```python
class Employee:
    def __init__(self, name: str) -> None:
        self.name = name
        self.department: Department | None = None


class Department:
    def __init__(self, name: str) -> None:
        self.name = name
        self.employees: list[Employee] = []

    def add(self, employee: Employee) -> None:
        self.employees.append(employee)
        employee.department = self  # keep both ends in sync


it = Department("IT")
for name in ["Kathy", "Fred", "Samantha"]:
    it.add(Employee(name))
print(len(it.employees), it.employees[1].department.name)
```

OUTPUT

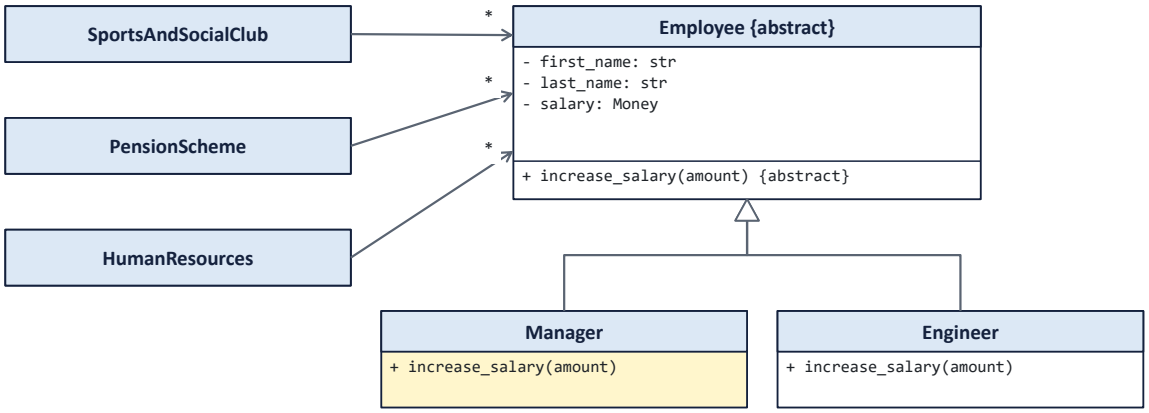
```
3 IT
```

Delegation

- Solve a problem by handing it to a **more cohesive** object or method
- Like people: a manager delegates to a skilled employee; you call a plumber
- A car delegates *accelerate*, *brake* and *steer* to its engine, brakes and wheels

You can delegate to…	Python example
a more cohesive linked object	`self.engine.start()`
a collection of linked objects	`for wheel in self.wheels: wheel.turn(angle)`
a method in a subclass	`self.area()` — overridden, chosen by polymorphism
a method in a superclass	`super().withdraw(amount)`
a method in the same class	`self._validate(amount)`

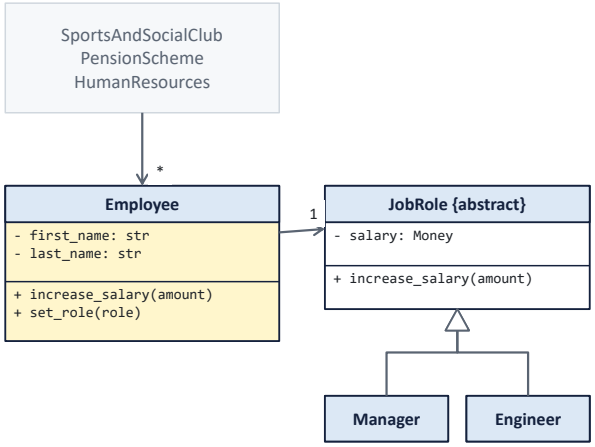
Delegation: The Problem



Watch out

Promoting Fred from `Engineer` to `Manager` means creating a **new object**: every association (club, pension, HR) must be found and re-linked, and Fred's identity is lost.

Delegation: The Solution



employee.py

```python
class Employee:
    def __init__(self, name: str,
                 role: JobRole) -> None:
        self.name = name
        self.role = role

    def set_role(self, role: JobRole) -> None:
        self.role = role  # promotion

    def increase_salary(self, pct: float) -> None:
        self.role.increase_salary(pct)  # delegate
```

Key idea

A promotion is `fred.set_role(Manager(...))`: the **same** `Employee` object, so every association stays valid.

Favoring Composition

Gang of Four

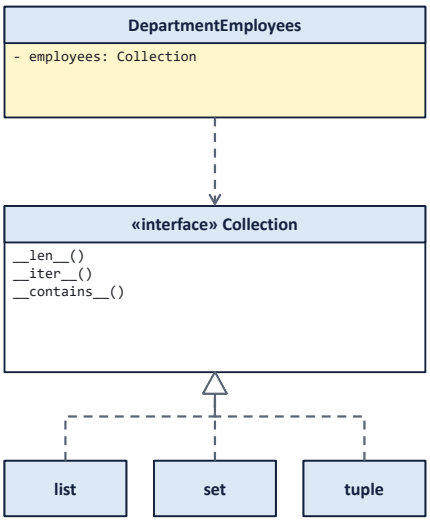
"Favor object composition over [implementation] inheritance." — Gamma, Helm, Johnson, Vlissides, *Design Patterns*

Their three principles: **favour composition** · **program to an interface** · **design for change**.

	Implementation inheritance	Composition
Reuse style	white-box: the subclass sees internals	black-box: only the public interface
Coupling	tight: superclass changes ripple down	loose: parts can be replaced
Decided	statically, in the `class` statement	at runtime; can change later
Exposes	the whole superclass interface	only what the whole chooses
Python example	`class Staff(list)`	`self._employees = []`

Programming to an Interface

“Program to an interface, not an implementation.” — Gamma, Helm, Johnson, Vlissides




```python
from collections.abc import Iterable


def total_salary(salaries: Iterable[float]) -> float:
    return sum(salaries)


print(total_salary([4000.0, 5000.0]))
print(total_salary((4000.0, 5000.0)))
print(total_salary({"Fred": 4000.0}.values()))
```

OUTPUT

```
9000.0
9000.0
4000.0
```

Tip

Depend on the most general type that provides what you need: `Iterable`, `Sequence`, `Mapping` or a `Protocol`.

DISCUSS

Inheritance or Composition?

For each pair, choose *inheritance*, *composition* or *interface* — and justify it:

- A `Stack` that offers `push`, `pop` and `peek` — built on `list`
- `Car` and `Engine`; `ElectricCar` and `Car`
- A `Logger` that writes to a file *or* the console, chosen at runtime
- `Employee` and `Manager` in a company where people get promoted

QUIZ

Quick Check: Cohesion

`ReportManager` loads data from a database, computes statistics, renders HTML and sends e-mails. What is the main design problem?

- **A** Tight coupling to the `object` class
- **B** Low cohesion — a Blob class
- **C** Too much polymorphism
- **D** Missing multiple inheritance

ANSWER

Quick Check: Cohesion

`ReportManager` loads data from a database, computes statistics, renders HTML and sends e-mails. What is the main design problem?

- **A** Tight coupling to the `object` class
- **B** Low cohesion — a Blob class ✓
- **C** Too much polymorphism
- **D** Missing multiple inheritance

★ **Why**

Four unrelated responsibilities in one class means low cohesion. Split it into `DataLoader`, `Statistics`, `HtmlRenderer` and `Mailer`, and compose them.

TRY IT

Refactor a Stack to Composition

```python
class Stack(list):
    def push(self, item: int) -> None:
        self.append(item)

    def peek(self) -> int:
        return self[-1]


stack = Stack()
stack.push(1)
stack.push(2)
stack.insert(0, 99)  # breaks the stack discipline!
print(stack.pop(), stack)
```

OUTPUT

```
2 [99, 1]
```

Rewrite `Stack` with **composition** so that only `push`, `pop`, `peek` and `len()` are available.

ANSWER

Refactor a Stack to Composition

```python
class Stack:
    def __init__(self) -> None:
        self._items: list[int] = []

    def push(self, item: int) -> None:
        self._items.append(item)

    def pop(self) -> int:
        return self._items.pop()

    def peek(self) -> int:
        return self._items[-1]

    def __len__(self) -> int:
        return len(self._items)


stack = Stack()
stack.push(1)
stack.push(2)
print(stack.pop(), stack.peek(), len(stack), hasattr(stack, "insert"))
```

OUTPUT

```
2 1 1 False
```

06

SOLID Principles I

Design smells, single responsibility and open/closed modules

- SOLID at a glance and the symptoms it fights
- SRP — the Single Responsibility Principle
- OCP — the Open/Closed Principle

SOLID at a Glance

	Principle	In one sentence
S	Single Responsibility	A class should have only one reason to change
O	Open/Closed	Extend behaviour by adding code, not by editing working code
L	Liskov Substitution	Subtypes must work wherever their base type is expected
I	Interface Segregation	Clients should not depend on methods they do not use
D	Dependency Inversion	Policies and details both depend on abstractions

Origin

Collected by Robert C. Martin (2000–2002); the acronym was suggested by Michael Feathers. The principles apply to classes, modules and functions alike.

Symptoms of a Rotting Design

Rigidity
One change forces a cascade of changes in dependent modules
Remedies: OCP, DIP

Fragility
A change breaks code in places with no obvious relation to it
Remedies: SRP, LSP

Immobility
Useful parts cannot be reused because they drag everything along
Remedies: DIP, ISP

Viscosity
Doing it the right way is harder than adding another hack
Remedies: OCP, SRP

! **Watch out**
Principles are tools, not laws: applying them where nothing changes creates another smell — **needless complexity**.

Symptoms after Martin, Agile Software Development: Principles, Patterns, and Practices (2002)

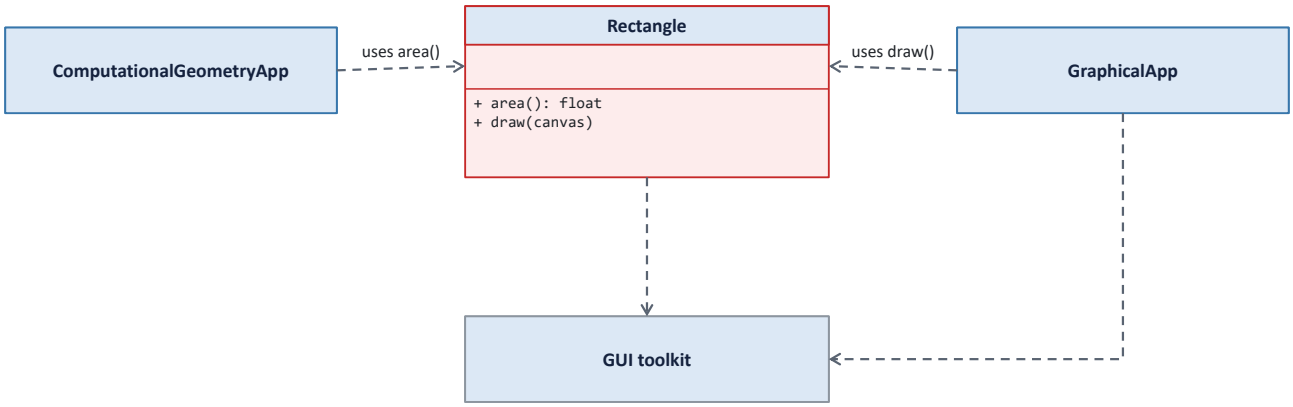
SRP: One Reason to Change

★ **Single Responsibility Principle (Martin)**
A class should have only one reason to change.

- A **responsibility** is a *reason to change* — an axis of change
- Two responsibilities in one class become **coupled**: changing one can break the other
- Roots: **cohesion**, the functional relatedness of a module's elements (DeMarco 1979; Page-Jones 1988)
- Later refinement: a module should be responsible to one **actor** (Martin 2017)

✗ **Common misconception**
SRP does not mean *one method per class* or *does one thing*: a `Fraction` class has many methods but only one reason to change — the arithmetic of fractions.

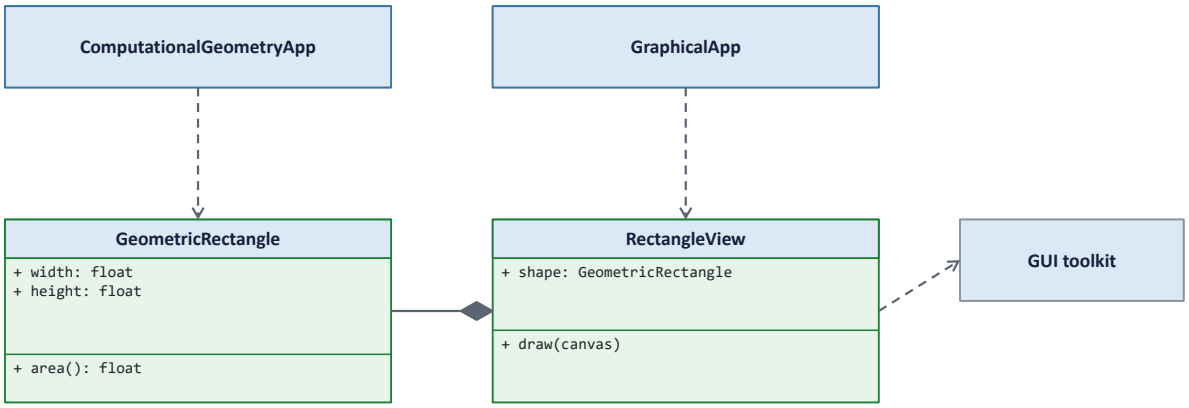
One Rectangle, Two Applications



Watch out

`Rectangle` models geometry **and** renders itself. The geometry application now depends on the GUI toolkit, and a GUI-driven change forces it to be retested and redeployed.

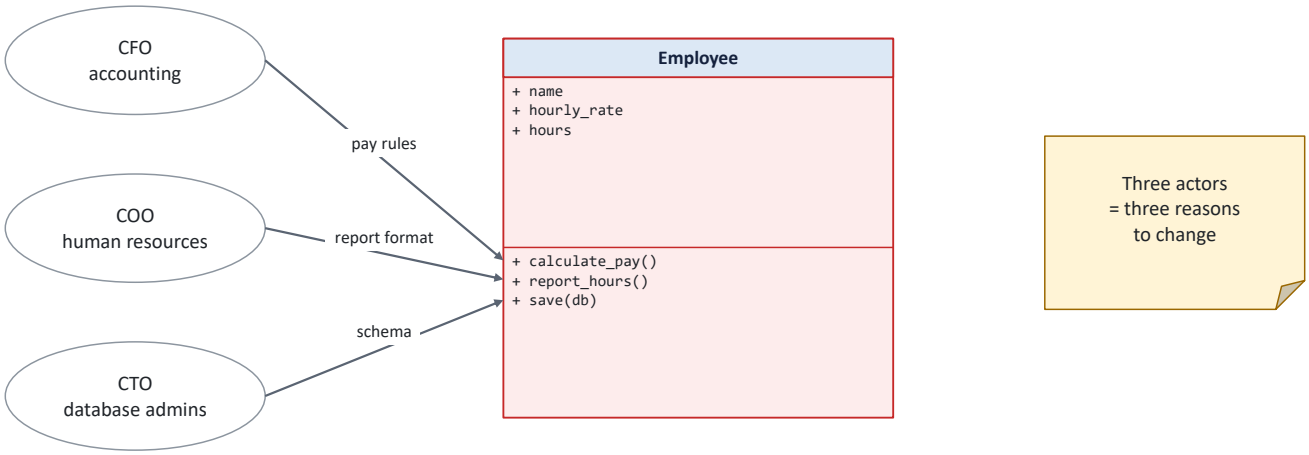
Separating Geometry from Rendering



Key idea

Each class now has one reason to change, and the geometry application no longer depends on the GUI at all.

Who Asks for the Change?



SRP Violation: The Do-Everything Employee

employee.py

```python
import sqlite3
from dataclasses import dataclass


@dataclass
class Employee:
    name: str
    hourly_rate: float
    hours: float

    def calculate_pay(self) -> float:               # accounting
        return self.hourly_rate * self.hours

    def report_hours(self) -> str:                  # HR
        return f"{self.name}: {self.hours:g} h"

    def save(self, db: sqlite3.Connection) -> None:  # DBAs
        db.execute("INSERT INTO employee VALUES (?, ?, ?)",
                   (self.name, self.hourly_rate, self.hours))
```

SRP Refactoring: One Class per Actor

`Employee` keeps only the data: a plain `@dataclass` with `name`, `hourly_rate` and `hours`.

```python
class PayCalculator:          # accounting
    def pay(self, emp: Employee) -> float:
        return emp.hourly_rate * emp.hours


class HoursReporter:          # HR
    def report(self, emp: Employee) -> str:
        return f"{emp.name}: {emp.hours:g} h"


ada = Employee("Ada", 50.0, 38)
print(PayCalculator().pay(ada))
print(HoursReporter().report(ada))
```

OUTPUT

```
1900.0
Ada: 38 h
```

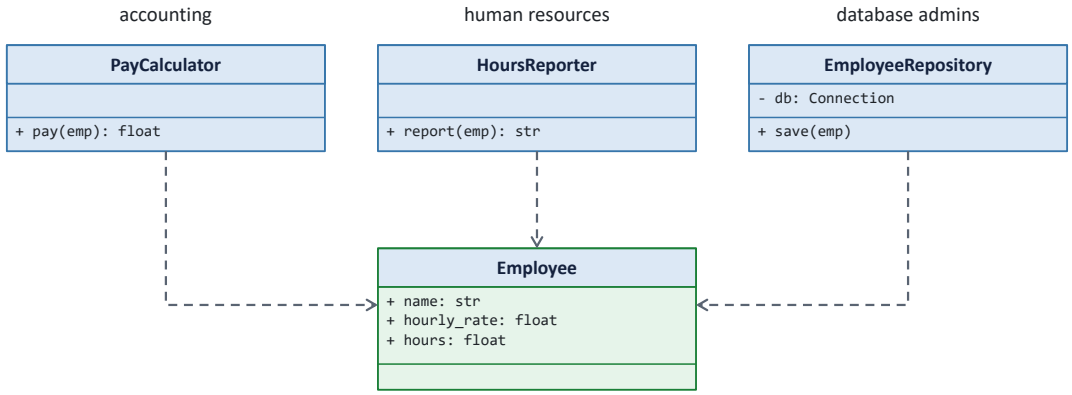
```python
class EmployeeRepository:     # DBAs
    def __init__(self, db: Connection) -> None:
        self.db = db

    def save(self, emp: Employee) -> None:
        self.db.execute(
            "INSERT INTO employee VALUES (?, ?, ?)",
            astuple(emp))
```

Key idea

Each class has one actor and one reason to change; a schema change can no longer break pay calculation.

The Refactored Employee Design



Key idea

The three classes depend on the shared data, never on each other — a change for one actor stays inside one class.

A Modem: One Responsibility or Two?

modem.py

```python
class Modem:
    def dial(self, phone_number: str) -> None: ...

    def hangup(self) -> None: ...

    def send(self, data: bytes) -> None: ...

    def receive(self) -> bytes: ...
```

Connection management

`dial` and `hangup` open and close the line

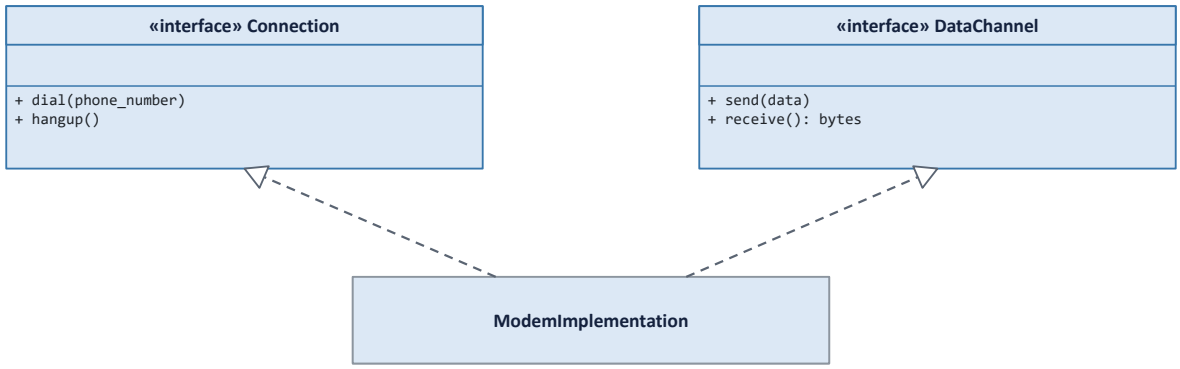
Data communication

`send` and `receive` move bytes over the line

It depends

Two groups of methods, used by different clients — but whether to split them depends on how the application changes.

Separate Only Along Real Axes of Change



- Split when the connection signatures change independently of data transfer
- Clients then depend on the interface they use, not on the whole modem
- If both always change together, splitting only adds **needless complexity**

6 · SOLID PRINCIPLES I

Spotting SRP Violations

The *and* test

Describe the class in one sentence. Needing *and* or *or* is a warning

Mixed imports

`sqlite3`, `smtplib` and pricing rules in one module

Change history

`git log` shows edits for unrelated reasons and by different teams

Painful tests

Testing one rule needs a database, a network or a GUI

★ **Key idea**

Conjoining responsibilities comes naturally; finding and separating them is much of what software design is about.

QUIZ

Quick Check: SRP

A `Report` class has `compute_totals()`, `to_html()` and `send_email()`. Which statement applies SRP best?

A. It is fine: all three methods are about reports

B. Split by reason to change: calculation, formatting and delivery change for different actors

C. Give every method its own class — one method per class

D. Merge them into one `process()` method so the class does one thing

ANSWER

Quick Check: SRP

A `Report` class has `compute_totals()`, `to_html()` and `send_email()`. Which statement applies SRP best?

A. It is fine: all three methods are about reports

B. Split by reason to change: calculation, formatting and delivery change for different actors

C. Give every method its own class — one method per class

D. Merge them into one `process()` method so the class does one thing

Why

SRP counts reasons to change, not methods. Totals change with business rules, HTML with presentation, email with infrastructure — three actors, three classes.

6 · SOLID PRINCIPLES I

OCP: Open for Extension, Closed for Modification

Open/Closed Principle (Meyer, 1988)

Software entities (classes, modules, functions, etc.) should be open for extension but closed for modification.

Open for extension

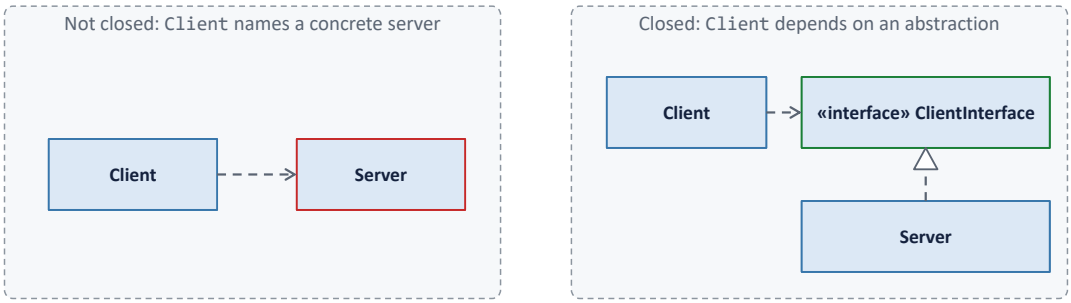
As requirements change, we can add new behaviour to the module

Closed for modification

Adding that behaviour does not edit the module's existing, tested source code

- Goal: the *next change of this kind* means **adding** code, not editing working code
- Symptom it cures: one change cascades through dependent modules (**rigidity**)

Abstraction Resolves the Paradox



- A module that depends only on a fixed abstraction is **closed** for modification
- New subclasses (or protocol implementations) **extend** its behaviour

Tip

Name the abstraction after the client's need (`ClientInterface`), not after its implementers (`AbstractServer`): abstractions belong to their clients.

OCP Violation: Switching on a Type Code

```python
def discounted_price(price: float, customer_type: str) -> float:
    if customer_type == "regular":
        return price
    elif customer_type == "student":
        return price * 0.8
    elif customer_type == "vip":
        return price * 0.7
    raise ValueError(f"unknown customer type: {customer_type!r}")


print(discounted_price(100.0, "student"))
```

```
OUTPUT
80.0
```

Watch out

A new `"employee"` type means editing this function — and every other switch on `customer_type`.

OCP Refactoring: Depend on a Policy Abstraction

checkout.py

```python
from abc import ABC, abstractmethod


class DiscountPolicy(ABC):
    """The fixed abstraction: an open-ended family of rules."""

    @abstractmethod
    def apply(self, price: float) -> float: ...


class Checkout:
    """Closed for modification: knows only the abstraction."""

    def __init__(self, policy: DiscountPolicy) -> None:
        self.policy = policy

    def total(self, prices: list[float]) -> float:
        return self.policy.apply(sum(prices))
```


Extending the Checkout Without Editing It

```python
class NoDiscount(DiscountPolicy):
    def apply(self, price: float) -> float:
        return price

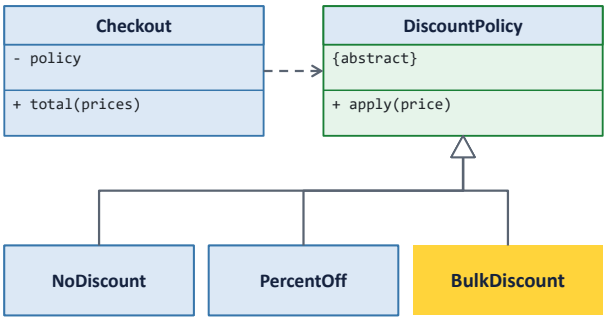

class PercentOff(DiscountPolicy):
    def __init__(self, percent: float) -> None:
        self.percent = percent

    def apply(self, price: float) -> float:
        return price * (100 - self.percent) / 100


cart = [40.0, 60.0]
print(Checkout(NoDiscount()).total(cart))
print(Checkout(PercentOff(20)).total(cart))
```

OUTPUT

```
100.0
80.0
```



added next year —
no edits above

Pythonic OCP: Functions as Policies

```python
from collections.abc import Callable

type Discount = Callable[[float], float]


def percent_off(percent: float) -> Discount:
    return lambda price: price * (100 - percent) / 100


def checkout_total(prices: list[float],
                   discount: Discount) -> float:
    return discount(sum(prices))


print(checkout_total([40.0, 60.0], percent_off(25)))
print(checkout_total([40.0, 60.0], lambda p: p - 5))
```

OUTPUT

```
75.0
95.0
```

Key idea

Any callable with the right signature is a new policy — no class needed. (The `type` alias statement needs Python 3.12+.)

Note

Standard-library OCP: `sorted(key=...)`, `logging.Handler` subclasses, `json.JSONEncoder.default`.

Strategic Closure

No module is closed against everything

Closing against one kind of change can make another kind harder

Close against likely change

Choose the axes you expect — or have already seen once

Signals

- The same `if`/`elif` on a type code in several places
- Each release adds one more variant

Cost

Every abstraction adds indirection; avoid speculative generality

Tip

A pragmatic rule: write the simple version first; when the first change of a new kind arrives, refactor so that the *next* one is an extension.

TRY IT

Open the Notifier for Extension

Next sprint adds a `"push"` channel. Refactor `notify` so that new channels need **no edits** to existing code.

```
def notify(message: str, channel: str) -> None:
    if channel == "email":
        print(f"EMAIL: {message}")
    elif channel == "sms":
        print(f"SMS: {message[:16]}")
    else:
        raise ValueError(f"unknown channel: {channel!r}")


notify("Server restarted at 02:00", "sms")
```

OUTPUT

```
SMS: Server restarted
```

Tip

Hint: one small class per channel behind a `Channel` protocol; the caller passes in the channels to use.

ANSWER

Open the Notifier for Extension

```
from typing import Protocol


class Channel(Protocol):
    def send(self, message: str) -> None: ...


class EmailChannel:
    def send(self, message: str) -> None:
        print(f"EMAIL: {message}")


class SmsChannel:
    def send(self, message: str) -> None:
        print(f"SMS: {message[:16]}")
```

```
class PushChannel:            # new: no edits
    def send(self, message: str) -> None:
        print(f"PUSH: {message}")


def notify(message: str,
           channels: list[Channel]) -> None:
    for channel in channels:
        channel.send(message)


notify("Server restarted at 02:00",
       [SmsChannel(), PushChannel()])
```

OUTPUT

```
SMS: Server restarted
PUSH: Server restarted at 02:00
```

07

SOLID Principles II

Substitutable subtypes, lean interfaces and inverted dependencies

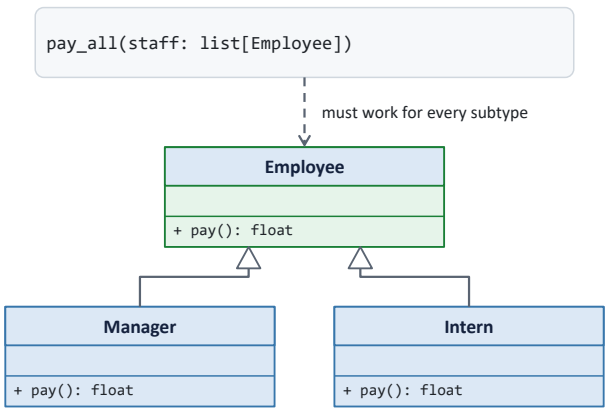
- LSP — the Liskov Substitution Principle
- ISP — the Interface Segregation Principle
- DIP — the Dependency Inversion Principle
- Applying SOLID together

LSP: Subtypes Must Be Substitutable

Liskov Substitution Principle (Liskov 1987; Martin)

Subtypes must be substitutable for their base types.

- If `X` **is-a** `Y`, then wherever a `Y` is used, an `X` must just work
- Neither Python nor type checkers can verify *behaviour* — the designer must
- A violation forces clients to use `isinstance` checks — and that breaks OCP



A Square Is a Rectangle… Isn't It?

shapes.py

```python
class Rectangle:
    def __init__(self, width: float,
                 height: float) -> None:
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height
```

The reasoning

Geometry says every square is a rectangle, so inheritance looks natural.

shapes.py (continued)

```python
class Square(Rectangle):
    """A rectangle whose sides stay equal."""

    def __init__(self, side: float) -> None:
        super().__init__(side, side)

    def _get_side(self) -> float:
        return self._side

    def _set_side(self, value: float) -> None:
        self._side = value

    # both names read and write the same side
    width = property(_get_side, _set_side)
    height = property(_get_side, _set_side)
```


TRACE THE CODE

Predict: Does the Square Pass?

```python
def check_area(rect: Rectangle) -> None:
    rect.width = 4
    rect.height = 5
    name = type(rect).__name__
    if rect.area() == 20:
        print(f"{name}: looking good")
    else:
        print(f"{name}: area {rect.area()}, expected 20!")


check_area(Rectangle(2, 3))
check_area(Square(10))
```

`check_area` was written for rectangles. Predict both lines of output.

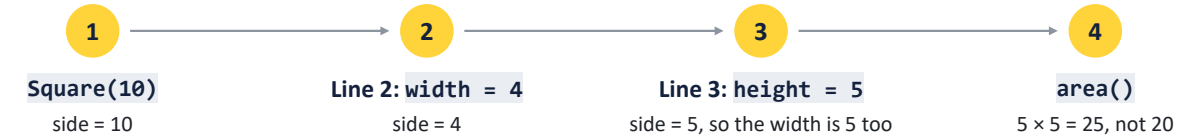
ANSWER

Predict: Does the Square Pass?

OUTPUT

```
Rectangle: looking good
Square: area 25, expected 20!
```

- Line 3 sets the shared side to 5 and silently overwrites the width
- `check_area` relies on a promise of `Rectangle`: setting `height` leaves `width` alone
- `Square` breaks that promise → it is **not substitutable** for `Rectangle`



`Square(10)`
side = 10

Line 2: `width = 4`
side = 4

Line 3: `height = 5`
side = 5, so the width is 5 too

`area()`
5 × 5 = 25, not 20

7 · SOLID PRINCIPLES II

Why Square Breaks the Rectangle Contract

What a client may assume about a `Rectangle`	`Rectangle`	`Square`
Setting `width` leaves `height` unchanged	✓	✗
`area()` equals `width * height`	✓	✓
After `width = 4; height = 5` the area is 20	✓	✗ (25)

Key idea

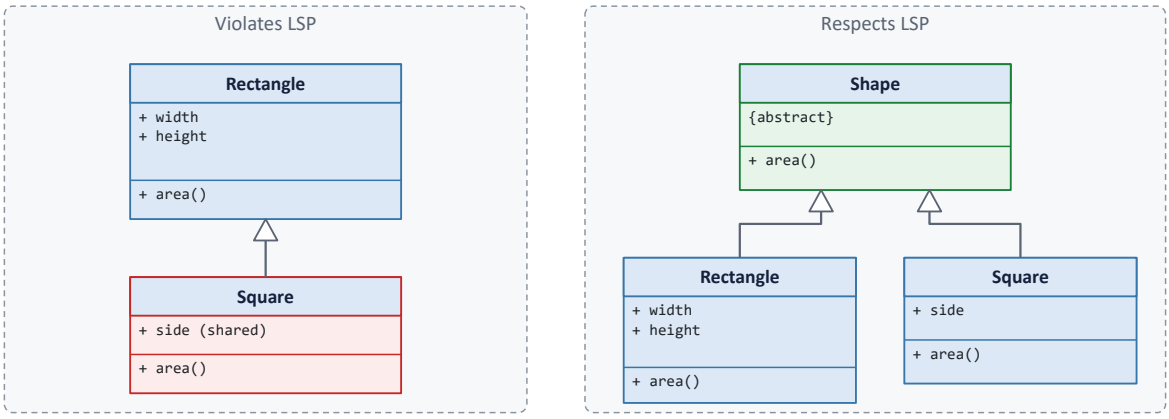
IS-A is about **behaviour**: a subtype must have no more constraints than its base type, because it must be usable everywhere the base type is.

Contract Rules for Subtypes

Rule for an overriding method	Violation example
Preconditions cannot be strengthened (accept at least as much)	`withdraw` suddenly rejects amounts above 100
Postconditions cannot be weakened (promise at least as much)	`Square` setters change the other side
Invariants of the base class must be preserved	a subclass allows a negative `balance`
No **new exception types** that callers do not expect	`fly()` raises `NotImplementedError` in `Penguin`
Signatures stay compatible (no extra required parameters)	`save(self, path, mode)` overrides `save(self, path)`

After Meyer's Design by Contract (1988) and Liskov & Wing (1994)

Fixing the Hierarchy: Siblings Under a Shape



Key idea

Clients that need to set width and height independently use `Rectangle`; clients that only need an area use `Shape`.

Substitutable Shapes in Code

shapes.py

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass


class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...


@dataclass
class Rectangle(Shape):
    width: float
    height: float

    def area(self) -> float:
        return self.width * self.height
```

shapes.py (continued)

```python
@dataclass
class Square(Shape):
    side: float

    def area(self) -> float:
        return self.side ** 2


shapes: list[Shape] = [
    Rectangle(5, 10), Square(3), Rectangle(2, 3)]
for shape in shapes:
    print(type(shape).__name__, shape.area())
```

OUTPUT

```
Rectangle 50
Square 9
Rectangle 6
```


QUIZ

Quick Check: LSP

`Account.withdraw(amount)` accepts any `0 < amount <= balance`, raises `InsufficientBalance` otherwise and returns the new balance. Which override **respects** LSP?

A. `FixedTermAccount.withdraw` always raises `PermissionError`

B. `StudentAccount.withdraw` rejects every amount above 100

C. `AuditedAccount.withdraw` writes an audit log entry, then behaves exactly like `Account`

D. `BonusAccount.withdraw` returns `None` instead of the new balance

ANSWER

Quick Check: LSP

`Account.withdraw(amount)` accepts any `0 < amount <= balance`, raises `InsufficientBalance` otherwise and returns the new balance. Which override **respects** LSP?

A `FixedTermAccount.withdraw` always raises `PermissionError`

B `StudentAccount.withdraw` rejects every amount above 100

C `AuditedAccount.withdraw` writes an audit log entry, then behaves exactly like `Account`

D `BonusAccount.withdraw` returns `None` instead of the new balance

★ **Why**

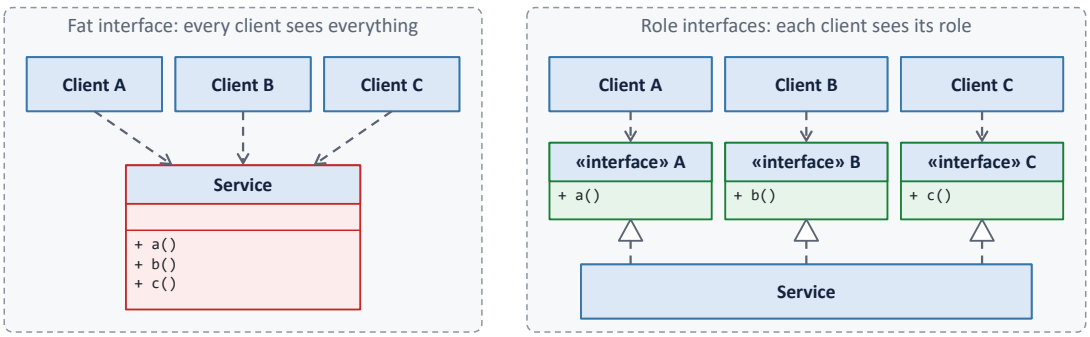
Adding behaviour that keeps the contract is fine. A raises a new exception type, B strengthens the precondition, D weakens the postcondition.

7 · SOLID PRINCIPLES II

ISP: Keep Interfaces Client-Specific

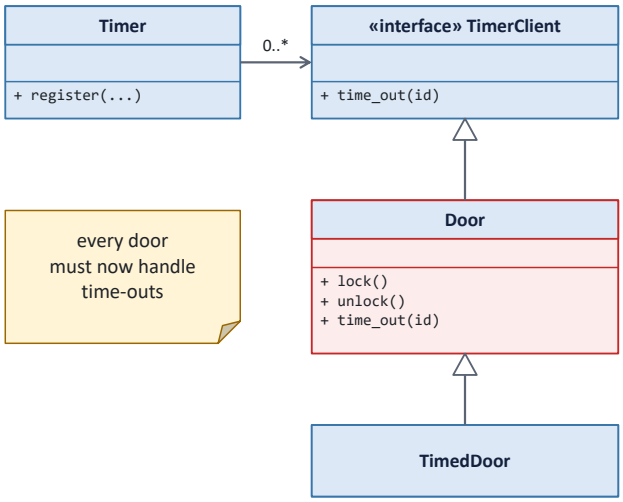
★ **Interface Segregation Principle (Martin)**

Clients should not be forced to depend on methods they do not use.



- A **fat interface** bundles method groups that serve different clients
- Prefer small, cohesive **role interfaces** — in Python, small ABCs or `typing.Protocol`s

Interface Pollution: The Timed Door



timer.py

```python
from abc import ABC, abstractmethod


class TimerClient(ABC):
    @abstractmethod
    def time_out(self, timeout_id: int) -> None: ...


class Timer:
    def register(self, timeout: float, timeout_id: int,
                 client: TimerClient) -> None:
        """Call client.time_out(timeout_id) later."""
```


Pollution Forces Every Door to Time Out

```python
class Door(TimerClient):
    def lock(self) -> None:
        print("locked")

    def unlock(self) -> None:
        print("unlocked")


class GardenGate(Door):  # a gate has no timer at all
    pass


GardenGate().lock()
```

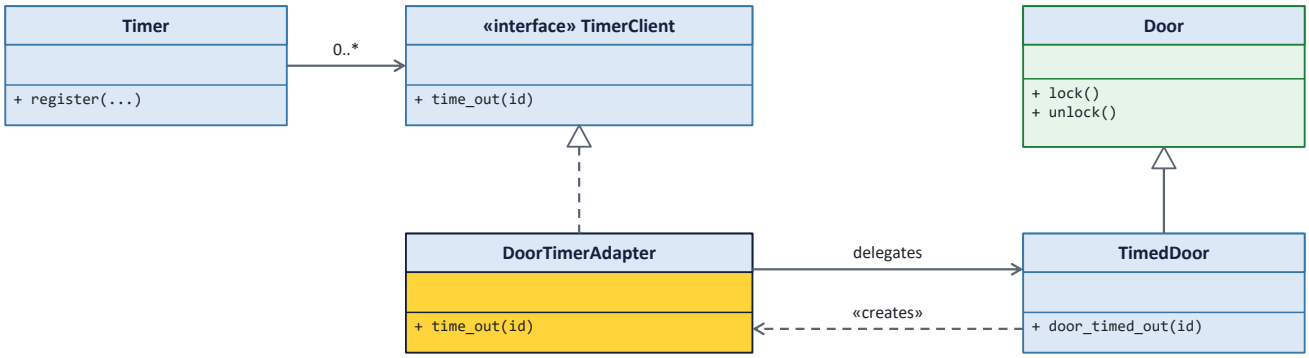
OUTPUT

```
Traceback (most recent call last):
  ...
TypeError: Can't instantiate abstract class GardenGate without an implementation for abstract method 'time_out'
```

Watch out

The only escape inside this design is a stub `time_out` that does nothing or raises — an LSP violation waiting to happen.

Separation Through Delegation



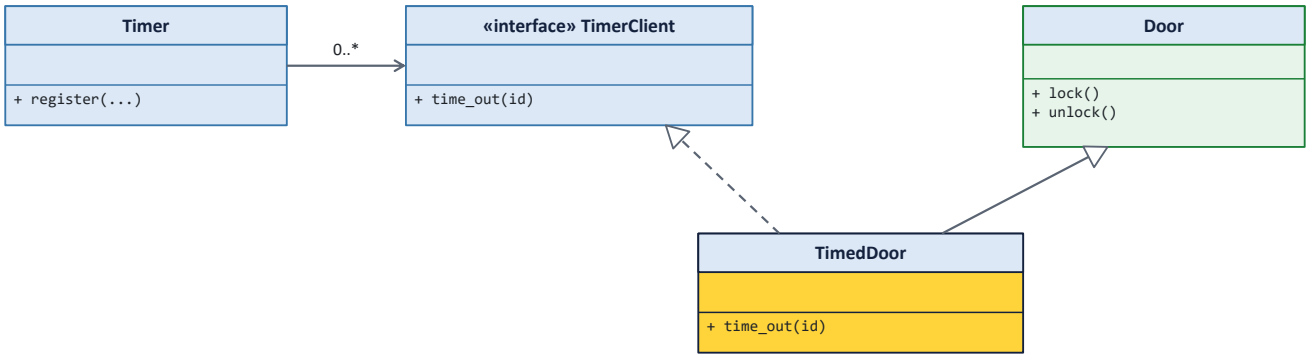

```python
class DoorTimerAdapter(TimerClient):
    def __init__(self, door: TimedDoor) -> None:
        self.door = door

    def time_out(self, timeout_id: int) -> None:
        self.door.door_timed_out(timeout_id)
```

Key idea

This is the **Adapter** pattern (Gamma et al., 1994): `Door` stays free of timer concerns.

Separation Through Multiple Inheritance




```python
class TimedDoor(Door, TimerClient):
    def time_out(self, timeout_id: int) -> None:
        self.lock()
```

- Clients of `Door` see only door methods
- The `Timer` sees only `TimerClient`
- No adapter object needed

ISP with `typing.Protocol`

timer.py

```python
from typing import Protocol


class TimerClient(Protocol):
    def time_out(self, timeout_id: int) -> None: ...


class Timer:
    def __init__(self) -> None:
        self._clients: dict[int, TimerClient] = {}

    def register(self, timeout_id: int,
                 client: TimerClient) -> None:
        self._clients[timeout_id] = client

    def expire(self, timeout_id: int) -> None:
        client = self._clients.pop(timeout_id)
        client.time_out(timeout_id)
```

doors.py

```python
class Door:
    def lock(self) -> None:
        print("door locked")

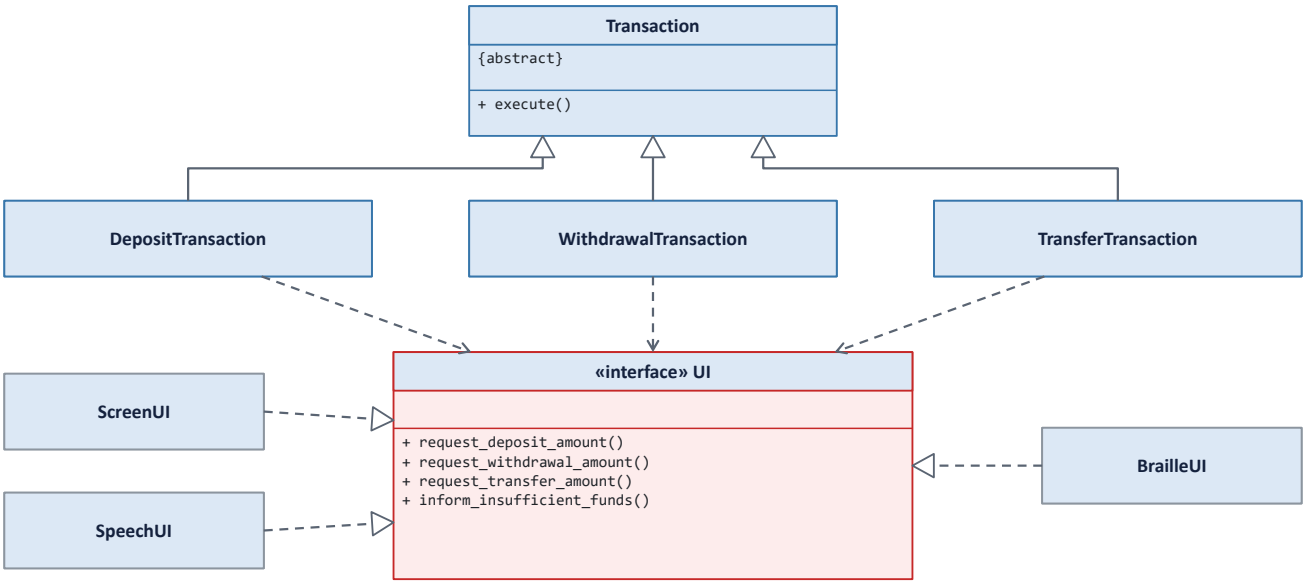

class TimedDoor(Door):  # no TimerClient base
    def time_out(self, timeout_id: int) -> None:
        print(f"timer {timeout_id} expired")
        self.lock()


timer = Timer()
timer.register(7, TimedDoor())
timer.expire(7)
```

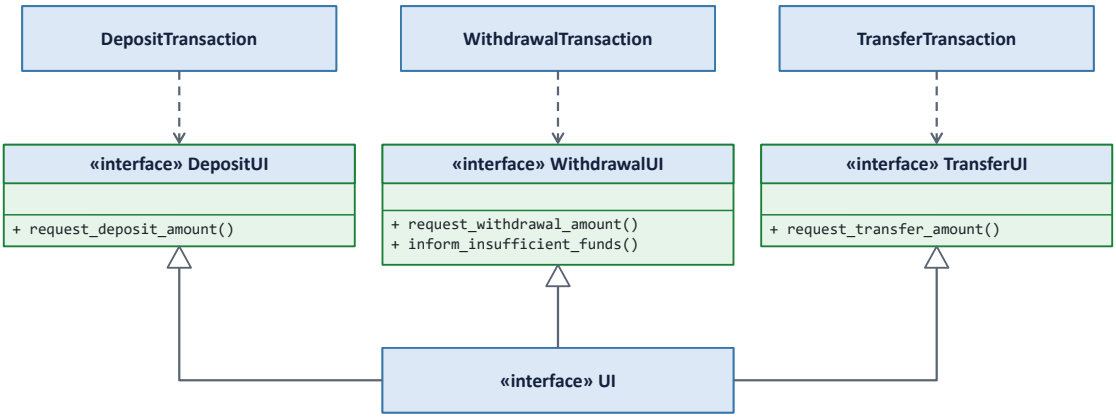
OUTPUT

```
timer 7 expired
door locked
```


The ATM Example: One UI for All Transactions



Segregating the ATM UI



Key idea

Each transaction depends only on its own role interface; `UI` combines the roles for the concrete screen, speech and Braille classes.

Role Interfaces Make Test Doubles Tiny

atm.py

```python
from typing import Protocol


class DepositUI(Protocol):
    def request_deposit_amount(self) -> float: ...


class DepositTransaction:
    def __init__(self, ui: DepositUI) -> None:
        self.ui = ui  # needs one role only

    def execute(self, balance: float) -> float:
        return balance + self.ui.request_deposit_amount()
```

test_atm.py

```python
class ScriptedUI:
    """Test double: a fixed answer."""

    def __init__(self, amount: float) -> None:
        self.amount = amount

    def request_deposit_amount(self) -> float:
        return self.amount


deposit = DepositTransaction(ScriptedUI(250.0))
print(deposit.execute(balance=1000.0))
```

OUTPUT

```
1250.0
```

QUIZ

Quick Check: ISP

`Printer` is an ABC with abstract `print_page()`, `scan()` and `fax()`. A `BasicPrinter` can only print. What does ISP suggest?

A. Implement `scan()` and `fax()` in `BasicPrinter` with `raise NotImplementedError`

B. Split into small role interfaces (`Printable`, `Scannable`, `Faxable`); clients depend on the role they use

C. Drop the interface and check `hasattr(device, "scan")` before every call

D. Make `BasicPrinter` inherit from `MultiFunctionPrinter` and disable the extras

ANSWER

Quick Check: ISP

`Printer` is an ABC with abstract `print_page()`, `scan()` and `fax()`. A `BasicPrinter` can only print. What does ISP suggest?

A. Implement `scan()` and `fax()` in `BasicPrinter` with `raise NotImplementedError`

B. Split into small role interfaces (`Printable`, `Scannable`, `Faxable`); clients depend on the role they use

C. Drop the interface and check `hasattr(device, "scan")` before every call

D. Make `BasicPrinter` inherit from `MultiFunctionPrinter` and disable the extras

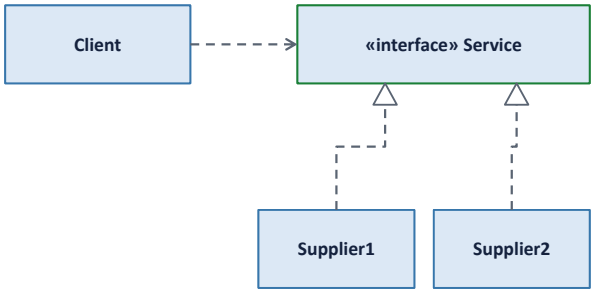
★ **Why**

Role interfaces let `BasicPrinter` implement only `Printable`. A and D create non-substitutable stubs (LSP), C scatters type checks through every client (OCP).

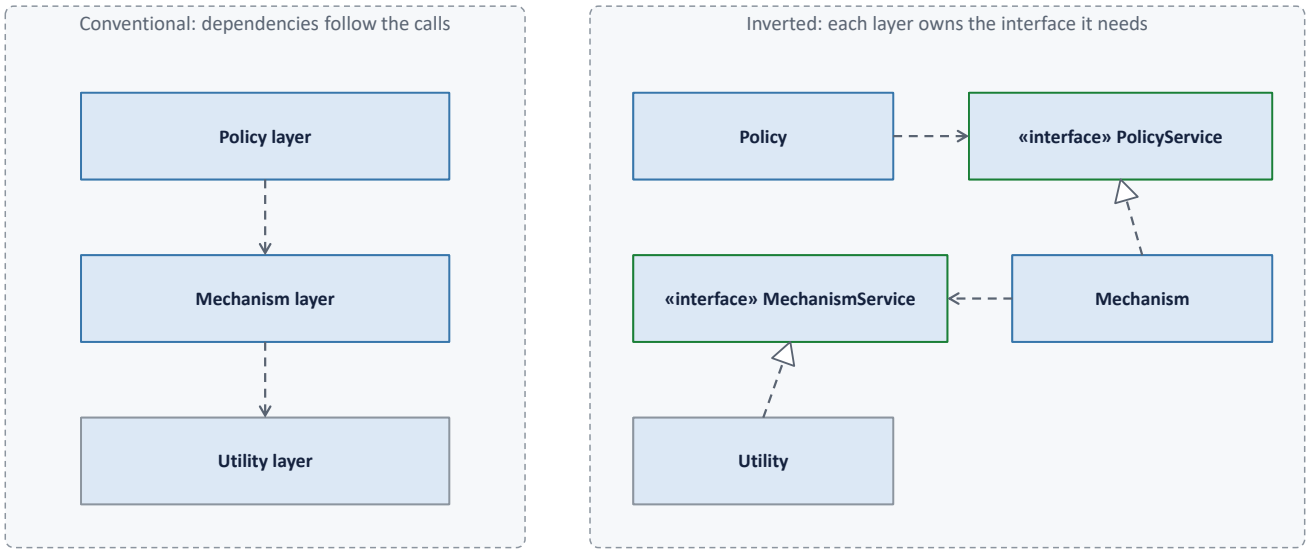
DIP: Depend on Abstractions

Knoernschild (2002)
Depend on abstractions. Do not depend on concretions.

- High-level modules should not depend on low-level modules — both depend on **abstractions**
- Abstractions should not depend on details — details depend on abstractions
- Python abstractions: an ABC or a `typing.Protocol`



High-Level Policy vs Low-Level Details



DIP Violation: A Policy Wired to Details

orders.py

```python
import smtplib
import sqlite3


class OrderService:
    """High-level policy, hard-wired to low-level details."""

    def __init__(self) -> None:
        self.db = sqlite3.connect("shop.db")
        self.smtp = smtplib.SMTP("mail.example.com")

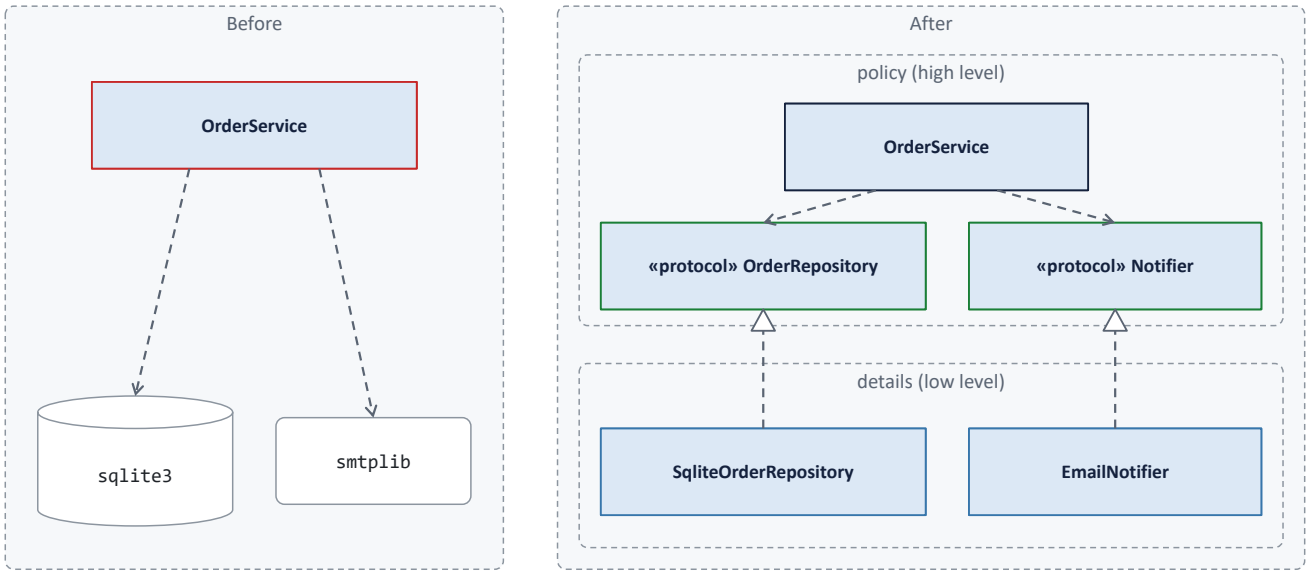
    def place_order(self, customer: str, total: float) -> None:
        self.db.execute("INSERT INTO orders VALUES (?, ?)",
                        (customer, total))
        self.smtp.sendmail("shop@example.com", customer,
                           f"Thanks! Total: {total:.2f}")
```

Watch out

Cannot test the business rule without a real database and mail server

Cannot switch to SMS or PostgreSQL without editing the business rule

Inverting the Dependencies



DIP Step 1: The Policy Owns Its Abstractions

orders.py

```python
from dataclasses import dataclass
from typing import Protocol


@dataclass(frozen=True)
class Order:
    customer: str
    total: float


class OrderRepository(Protocol):
    def save(self, order: Order) -> None: ...


class Notifier(Protocol):
    def notify(self, recipient: str,
               message: str) -> None: ...
```

orders.py (continued)

```python
class OrderService:
    """High-level policy: no SQL, no SMTP."""

    def __init__(self, repository: OrderRepository,
                 notifier: Notifier) -> None:
        self.repository = repository
        self.notifier = notifier

    def place_order(self, order: Order) -> None:
        self.repository.save(order)
        self.notifier.notify(
            order.customer,
            f"Thanks! Total: {order.total:.2f}")
```

Key idea

`orders.py` imports no database and no mail library.

DIP Step 2: Details Implement the Abstractions

infrastructure.py

```python
from sqlite3 import Connection


class SqliteOrderRepository:
    def __init__(self, db: Connection) -> None:
        self.db = db

    def save(self, order: Order) -> None:
        self.db.execute(
            "INSERT INTO orders VALUES (?, ?)",
            (order.customer, order.total))


class ConsoleNotifier:  # stands in for e-mail
    def notify(self, recipient: str,
               message: str) -> None:
        print(f"to {recipient}: {message}")
```

main.py

```python
import sqlite3


def main() -> None:
    """Composition root: choose and wire details."""
    db = sqlite3.connect(":memory:")
    db.execute("CREATE TABLE orders (customer, total)")
    service = OrderService(SqliteOrderRepository(db),
                           ConsoleNotifier())
    service.place_order(Order("ada@example.com", 42.5))


main()
```

OUTPUT

```
to ada@example.com: Thanks! Total: 42.50
```

Injecting Dependencies

Technique	Looks like	Use it for
Constructor injection	`OrderService(repo, notifier)`	required, long-lived collaborators (default choice)
Method injection	`report.render(formatter)`	a collaborator that varies per call
Default + override	`clock: Clock | None = None`	a sensible production default, replaced in tests
Hard-wired creation	`self.db = sqlite3.connect(...)`	avoid inside policies

Tip

Create concrete objects in one **composition root** (`main()`). Python rarely needs a DI container; frameworks such as FastAPI provide `Depends` (Module 8).

DIP Enables Testing with Fakes and Mocks

test_orders.py

```python
class InMemoryOrderRepository:
    """A fake: real behaviour, no database."""

    def __init__(self) -> None:
        self.orders: list[Order] = []

    def save(self, order: Order) -> None:
        self.orders.append(order)
```

Note

Fakes and mocks are **test doubles** — Module 3 covers them with `pytest`.

```python
from unittest.mock import Mock


def test_place_order_notifies() -> None:
    repository = InMemoryOrderRepository()
    notifier = Mock()   # records every call
    service = OrderService(repository, notifier)

    service.place_order(Order("bo@uni.edu", 10.0))

    assert repository.orders == [
        Order("bo@uni.edu", 10.0)]
    notifier.notify.assert_called_once_with(
        "bo@uni.edu", "Thanks! Total: 10.00")


test_place_order_notifies()
print("test passed")
```

OUTPUT

```
test passed
```

When to Invert — and When Not

Invert volatile details

Databases, network and e-mail, file system, clock, randomness, third-party APIs

Do not wrap the stable

`str`, `list`, `math`, `dataclasses`: depending on them directly is fine

Let the client shape it

Protocols speak the policy's language: `save(order)`, not `execute(sql)`

One composition root

Concrete classes are chosen and wired in one place, near `main()`

Note

ABC or `Protocol`? ABCs enforce implementation at instantiation; Protocols need no inheritance and suit third-party classes. Both invert the dependency.

QUIZ

Quick Check: DIP

In the order example, where should the `Notifier` protocol be defined?

A. In the e-mail module, next to `EmailNotifier`

B. In the high-level module, next to `OrderService` which uses it

C. In a separate third-party package installed with `pip`

D. Nowhere: with duck typing, Python needs no abstractions

ANSWER

Quick Check: DIP

In the order example, where should the `Notifier` protocol be defined?

- A — In the e-mail module, next to `EmailNotifier`
- **B** — In the high-level module, next to `OrderService` which uses it
- C — In a separate third-party package installed with `pip`
- D — Nowhere: with duck typing, Python needs no abstractions

Why

The client owns the abstraction. If `Notifier` lived with `EmailNotifier`, the policy would import the detail module and the dependency would point the wrong way again.

TRY IT

Make the Greeter Testable

```python
from datetime import datetime


class Greeter:
    def greet(self, name: str) -> str:
        hour = datetime.now().hour      # hidden dependency
        part = "morning" if hour < 12 else "afternoon"
        return f"Good {part}, {name}!"
```

- Define a `Clock` protocol with a `now()` method
- Inject the clock through `__init__`
- Show with a fixed clock at 09:00 that `greet("Ada")` says good morning

ANSWER

Make the Greeter Testable

greeter.py

```python
from datetime import datetime
from typing import Protocol


class Clock(Protocol):
    def now(self) -> datetime: ...


class Greeter:
    def __init__(self, clock: Clock) -> None:
        self.clock = clock

    def greet(self, name: str) -> str:
        hour = self.clock.now().hour
        part = "morning" if hour < 12 else "afternoon"
        return f"Good {part}, {name}!"
```

demo.py

```python
class SystemClock:                # production
    def now(self) -> datetime:
        return datetime.now()


class FixedClock:                 # test double
    def __init__(self, moment: datetime) -> None:
        self.moment = moment

    def now(self) -> datetime:
        return self.moment


nine_am = FixedClock(datetime(2026, 9, 28, 9, 0))
print(Greeter(nine_am).greet("Ada"))
```

OUTPUT

```
Good morning, Ada!
```


7 · SOLID PRINCIPLES II

SOLID Together: A Design Review Checklist

Symptom in the code	Principle	Typical refactoring
One class changes for unrelated reasons	SRP	Extract one class per actor
Every new variant edits an `if`/`elif` chain	OCP	Polymorphism or a callable policy
Overrides refuse, stub or surprise; callers use `isinstance`	LSP	Repair the hierarchy; prefer composition
Implementers write stubs for methods they do not need	ISP	Split into role interfaces
Business logic imports database or network clients	DIP	Client-owned protocol + injection

Key idea

The principles reinforce each other: small classes (SRP) with small interfaces (ISP), used through abstractions (DIP) that subtypes honour (LSP), give extensible code (OCP).

Practice Problems

SRP · Split `UserManager`

It validates input, hashes passwords, stores users in SQLite and sends welcome e-mails. Find the actors and split it

OCP · Shipping costs

`shipping_cost(order, method)` switches on `"standard"`, `"express"`, `"drone"`. Make new methods pluggable

LSP · `Stack(list)`

A `Stack` subclasses `list` and forbids `insert`. Which client code breaks? Redesign it with composition

ISP · Smart devices

A `SmartDevice` ABC requires `play_music`, `make_call`, `take_photo`. Let a `SmartSpeaker` implement only what it can

DIP · Lottery draw

`LotteryDraw` calls `random.sample` and `datetime.now` directly. Inject both and test with fakes

Review · Bank accounts

Apply all five principles to the `Account` example from earlier sections. Which change does each one make easier?

✓ **How to work**

For each problem: name the **symptom**, then the **principle**, then the **refactoring** — and prove it with a small runnable test.

Key Takeaways

- ✓ Objects combine state, behaviour and identity; classes define them, and properties protect their invariants
- ✓ Exceptions are raised, handled and propagate; a custom hierarchy makes failures explicit
- ✓ Inheritance models *is-a*; polymorphism, ABCs and `Protocol`s work with any conforming type
- ✓ Good design: high cohesion, low coupling — favour composition, program to an interface
- ✓ SRP and OCP: one reason to change per class; extend by adding code, not by editing it
- ✓ LSP, ISP and DIP: substitutable subtypes, lean role interfaces, policies that own their abstractions

NEXT UP Module 02 — Functional Programming in Python